

Introduction to Section

Agile Overview

End of lecture

Summing Agile Core Principles

Welcome and expect change

Create small value-added increments

Utilize feedback loops

Constantly learn through discovery and creativity

Focus on developing a value for stakeholders

It's okay to fail fast

Deliver value throughout the project

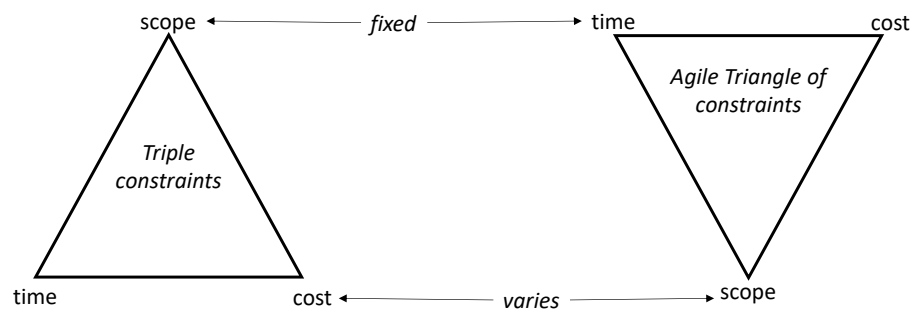
Constantly improve upon the project and the project work

Agile Approaches and Methods

- Agile approaches and agile methods are umbrella terms that cover a variety of frameworks and methods.
 - Lean – Kanban
 - Agile
 - Scrum
 - eXtreme Programming (XP)
 - Feature-Drive Development (FDD)
 - Dynamic Systems Development Method (DSDM)
 - Agile Unified Process (AUP)
 - Crystal versions

Inverted Triangle Model

- Time/cost/scope
- What's a variable and what is fixed



Agile Mindset

End of lecture

The Agile Mindset

- What does it mean to “BE AGILE”?
- Being agile isn’t simply a matter of using a certain set of tools or practices or following a specific methodology
- Agility involves a new mindset – way of thinking – based on the Agile Manifest including values and principles

Doing Agile Versus Being Agile

Doing Agile

- Agile practices
 - Iterations
 - Stand-up meetings
- No agile mindset

Being Agile

- Agile mindset
- Embracing agile principles
- Tailoring processes

Personal

- You, the project manager, must:
 - Embrace agile mindset
 - Understand agile principles
 - Sell the idea of agile
 - Expect skeptics

Team

- Agile team members must:
 - See the value of agile approach
 - Experience benefits agile provides
 - Have quick victories

Stakeholders

- Project stakeholders must:
 - Be willing to try agile approach
 - Be convinced on agile values
 - Experience quick ROI
 - Eat dessert first

Agile Principles and Mindset Tasks

- Advocate for agile principles and values in the organization
- Ensure common understanding of agile principles
- Educate and influence agile
- Transparency equates to trust
- Safe environment for experimenting

Agile Principles and Mindset Tasks

- Experiment with new techniques and processes
- Share knowledge new collaboration
- Emergent leadership
- Practice servant leadership

Agile Values and Principles

- There are two strategies to fulfill agile values and principles:
- Adopt a formal agile approach, designed and proven to achieve desired results.
- Implement changes to project practices in a manner that fits the project context to achieve progress on a core value or principle.

Utilizing Agile

- Why use agile?
- Different types of projects use different approaches
- Predictive project plan up front
- Knowledge work has many unknowns

Can you spot the problem? Yes - water flows downhill! It wasn't easy for later stages to provide input or valuable feedback into previous ones. Although there was typically some overlap the specialists responsible for a given stage were often moved on to their next project once their stage was completed.

What if a requirement was missed and only identified during testing?
What if a deployment issue required changes to the system design?
What if testing highlighted a misunderstanding with requirements?

Agile Manifesto

End of lecture

The Agile Manifesto

- The Agile Manifesto includes four value statements and twelve guiding principles.
- Written by software development experts.
- It uses terms from the IT field to express ideas.

Individuals And Interactions Over Processes and Tools

- Individuals and interactions are most important
- Processes and tools will be needed on projects
- Projects are completed by people not processes and tools
- Agile projects are people driven

As detailed processes and sophisticated tools were developed, the focus shifted from the people to the tools and process. Vendors would sell a process or tool as a solution. All along, it was still people. What this point means to me is that the focus should be on the people and the communication between them. The process and tools should be the minimum needed for a given situation.

Focus on People First

- People complete projects
- People negotiate
- People do knowledge work
- Processes create repeatable results
- Agile is not a machine

Working Software Over Comprehensive Documentation

- Agile project need to deliver value
- Value is about the purpose or business need the project aims to deliver
- Documentation is barely sufficient
- Documentation is done just in time – as the last responsible moment
- Documentation might also be just because
 - Industry requirements
 - Organizational requirements

So what level of documentation is proper? The philosophy of Agile is to do the least amount of something and no more for a given situation. A regulated industry will require much more documentation and trace matrices than an unregulated one. Agile process can accommodate this whole spectrum of needs.

Beware Documentation

- Documentation adds no value
- Who reads the EULA?
- Who reads the reports?
- Comments are okay, excessive documentation is waste

Customer Collaboration Over Contract Negotiation

- Agile is flexible, accommodating, and willing to change
- Contracts are often rigid and uncooperative
- Agile contracts must accommodate change
- Difference between being right and doing the right thing

When we focus on a contract, we get a false sense of security and think everyone agrees to what is being done. After all, we have a signed contract, so we must be OK. In reality, we do not know what we do not know. An iterative approach is the best solution to this problem.

We need contracts. They just need to be flexible and have the proper change control built in to accommodate an Agile process. There is a "sales" effort that's required to convince the customer of the benefits of a more "flexible" contract approach. We need to explain that there will be many opportunities for change and that the customer will be in complete control.

Contracts and Agile Projects

- Agile is uncertain
- Contracts are legal documents
- Contracts want certainty
- Define project approach
- Nature of agile to change

Responding to Change Over Following A Plan

- Agile welcomes change
- Predictive projects plan everything in advance
- Agile projects have lots and lots of many changes
- Agile projects have uncertainty up front

Many projects that have elaborate project plans with detailed Gantt charts have failed. We make the plans so complex and detailed that they're difficult to modify when changes occur. Agile process replaces project plans with release schedules and burn-down charts that can accommodate change. We can still track progress and, in fact, progress is more transparent than in a typical Waterfall project plan.

9 Keys of Great Agile Projects

End of lecture

Business people and developers
must work together daily
throughout the project.

1

Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

2

The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

3

Working software is the
primary measure of
progress.

4

Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

5

Continuous attention to
technical excellence and
good design enhances agility.

6

Simplicity--the art of
maximizing the amount of
work not done--is essential.

7

The best architectures, requirements,
and designs emerge from self-
organizing teams.

8

At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

9

Diving Deeper: Scrum Basics

End of lecture

Scrum Details

- Evolved since 1990s
- Framework for processes, not a process itself
- Characteristics of product management, not just project management
- Rules and Roles of Scrum is what you need to know

Scrum Utilizations

- Research and identify viable markets, technologies, and product capabilities
- Develop products and enhancements
- Frequently release products and enhancements
- Develop and sustain Cloud and operations for product use
- Sustain and renew products

Theory of Scrum

- Empirical process control theory, or empiricism
- Empiricism asserts that knowledge comes from experience and making decisions based on what is known
- Scrum employs an iterative, incremental approach to optimize predictability and control risk

Three Scrum Pillars: TIA

- Transparency
- Inspection
- Adaptation

Transparency

- Transparency requires a common standard
- Observers share a common understanding of what is being seen
 - A common language shared by all participants
 - A common definition of “Done” (DoD)

Inspection

- Frequently inspect Scrum artifacts and progress
- Inspection should not get in the way of the work
- Most beneficial when performed by skilled inspectors during the work

Adaptation

- If the resulting product will be unacceptable, the process or the material being processed must be adjusted
- Adjustments are made ASAP to minimize additional deviations
- Scrum prescribes four formal events for inspection and adaptation:
 - Sprint Planning
 - Daily Scrum
 - Sprint Review
 - Sprint Retrospective

Agile, Predictive, or Hybrid?

End of lecture

Knowledge Work Projects are Special

- Industrial work requires up-front planning
- Knowledge work expects change
- Knowledge work is invisible work
- Agile is best suited for software development projects

Industrial Vs. Knowledge Work Projects

Industrial projects

- Visible
- Stable
- Running things
- Structure
- Correct answers
- Task driven
- Command and control
- Standards
- Performance measurement
- Cost of workers for a task

Knowledge work projects

- Invisible
- Lots of changes
- Changing environment
- Less structure
- Lots of questions
- Value-driven
- Autonomy driven
- Innovation
- Learning and teaching
- Workers are an asset not a cost

Definable vs. High-Uncertainty Work

- Definable work projects have clear procedures that have proven successful
- High-uncertainty projects have high rates of:
 - Change
 - Complexity
 - Risk

Defined Processes

- Repeatable processes
- Do the work the same way each time
- Prescribed process of action

Empirical Processes

- Based on experience and observation
 - Transparency
 - Inspection
 - Adaptation

Defined Processes vs. Empirical Processes

- Industrial work relies on defined processes
- Knowledge work relies on empirical processes
- A defined process defines all steps in advance

Defined Processes vs. Empirical Processes

- Empirical processes are:
 - Interactive
 - Incremental
 - Change often
 - Adapt
 - Pass through reviews

Incremental and Iterative Life Cycles

End of lecture

Iterative and Incremental Approaches

- Iterative and incremental approaches reduce waste and rework because of feedback. These approaches use:
- Very short feedback loops
- Frequent adaptation of process
- Reprioritization
- Regularly updated plans
- Frequent delivery

Core Principles of Agile

- Let's sum up the core principles of agile:
- Welcome change
- Work in small value-added increments
- Use build and feedback loops
- Learn through discovery
- Value-driven development
- Continuous delivery
- Continuous improvement

Four Types of Life Cycles

- Predictive Life Cycle
 - Fixed requirements
 - Activities performed once
 - Single delivery
 - Manage cost



Four Types of Life Cycles

- Agile Life Cycle

- Requirements are dynamic
- Activities repeated until correct
- Frequent small deliveries
- Goal is customer value via feedback



Four Types of Life Cycles

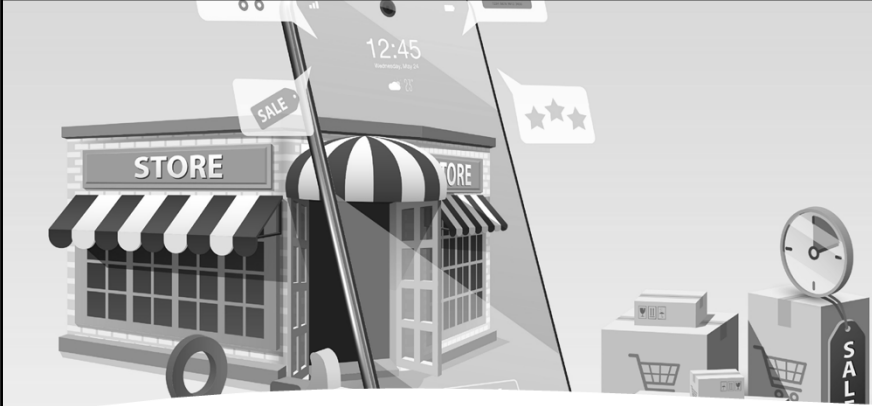
- Iterative Life Cycle
 - Requirements are dynamic
 - Activities repeated until correct
 - Single delivery
 - Correctness of solution



Four Types of Life Cycles

- Incremental Life Cycle
 - Requirements are dynamic
 - Performed once per increment
 - Frequent smaller deliveries
 - Speed





Shopping Online

Lorem ipsum dolor sit amet, consectetur
adipiscing elit, sed do eiusmod tempor
incididunt ut labore et dolore
magna aliqua.

■ ■ ■

Buy now

Incremental Delivery

- The team regularly deploys working increments
- Usually to a test environment for evaluation
- This is an opportunity for an early return on investment



Shopping Online

12:45
Wednesday, May 26

SALE

STORE

STORE

★ ★ ★

SALE

Buy now

Incremental Model

- System is broken down into many mini development projects
- Partial systems are built to produce the final system
- First tackle highest priority requirements
- The requirement of a portion is frozen once the incremented portion is developed



Pros: Incremental Delivery

- After each iteration, regression testing needed
 - Defects identified because few changes are made within any single iteration
- Allows for more targeted testing of each element
- Customer can respond to features and review the product
- Initial product delivery is faster and costs less



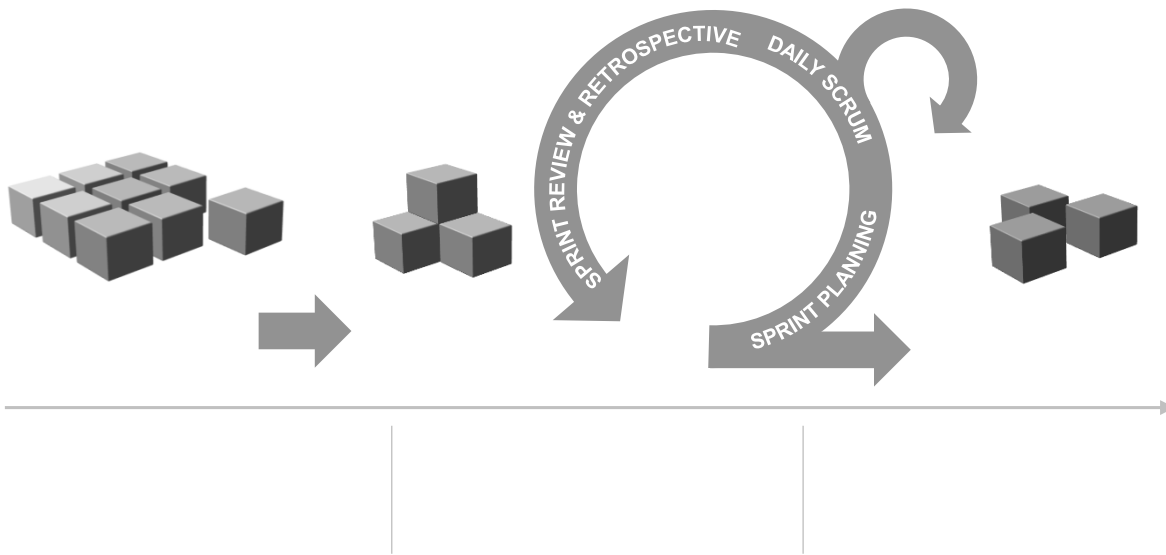
Cons: Incremental Delivery

- Cost may exceed business value
- Added functionality can reveal problems related to system architecture which were not evident earlier

Walking through a sprint

End of lecture

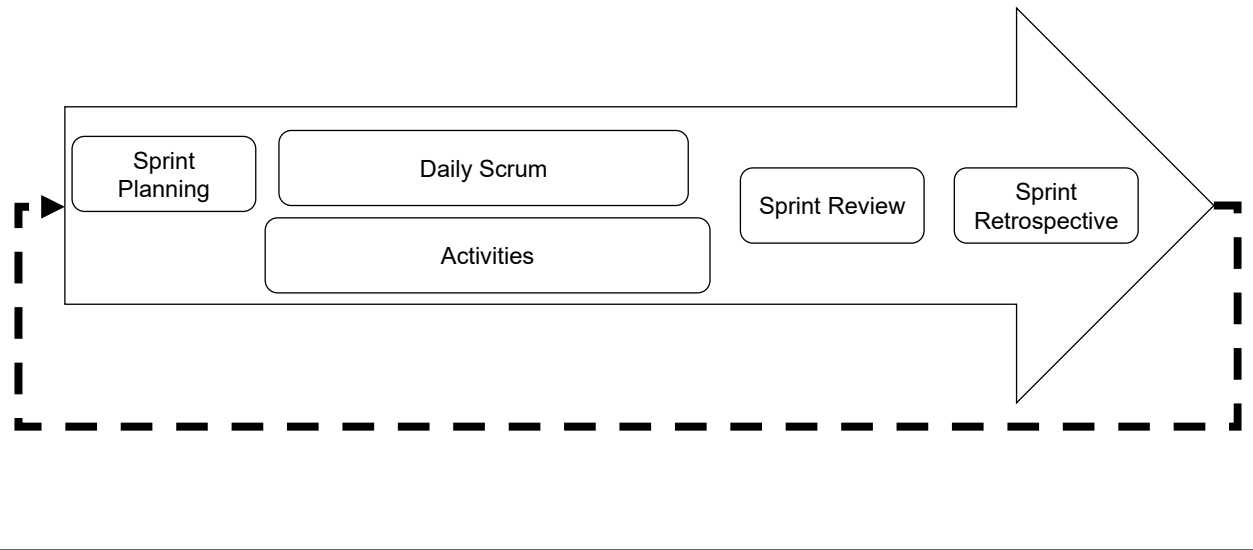
AGILE PROCESS



Pre-Sprint Activities

- Vision Statement: concise description of the goals of the project
- Product Roadmap: visual timeline of major product features to be delivered and is normally created by the Product Owner
- Stories: requirements normally written by the Product Owner and come from customer requirements
- Product Backlog: made up of stories and prioritized

Running a Sprint



Sprint Details

- Delivers final product after several Sprints
- Increment is developed in each Sprint
- Increment is a potentially releasable part of the final product
- Increment is a sum of all Product Backlog items completed so far

Sprint Details

- Increment increases after each Sprint
- Customers usually request changes during the Sprint Review
- Sprint Backlog doesn't change once the Sprint is started
- Sprint Goal should not change

Sprint

- Each *story* in the Product Backlog should normally be developed in a single Sprint
- Product Owner and Development Team select items from the top of the prioritized Product Backlog
- Must agree on a *Definition of Done* at the beginning of the project

Sprint

- Sprint Time boxes: usually 2 to 4 weeks
- Sprints are no more than one calendar month
- Avoid short sprints as goal is to avoid splitting a single item among several Sprints

Cancelling a Sprint

- Product Owner has the authority to cancel a Sprint
- Can happen when the Sprint Goal becomes obsolete
 - Changes in the Product Backlog, strategies, or approach
- When a Sprint is cancelled, the items that are “Done” will be reviewed and accepted, and the rest of the items will be put back into the Product Backlog

Agile Sprint Planning

End of lecture

Sprint Planning Meeting

- Timebox: Eight hours for a four-week sprint; less for shorter sprints
- Attendees: Complete Scrum team, including all roles
- Goal: Team capacity, Sprint Goal/Definition of Done, Sprint Backlog

Sprint Planning

- Development Team: no waiting for completed Product Backlog
- When Product Backlog has the necessary number of stories, Product Owner and Development Team can start the first Sprint
- First action is Sprint Planning
 - Time-boxed meeting
 - Usually fixed to 8 hours for a one-month Sprint
 - All three roles should attend this meeting
- Development Team estimates their capacity of for a Sprint

Sprint Planning

- Product Owner: ranked and ordered the Product Backlog
- Product Owner: user stories are easy to understand
- Development Team: selects appropriate number of items from the top of the Product Backlog, and puts them in the Sprint Backlog
- Development Team estimates effort for stories
- Selected Stories should equate to Development Team's capacity

Sprint Planning

- After Sprint backlog, Scrum Team drafts a Sprint Goal
 - Objective that should be met within the Sprint
 - Provides guidance to the Development Team on why it is building the Increment
- Sample Sprint Goal:
 - We are going to enable all the essential parts of the website catalog to set up a complete purchase process. This goal allows customers to begin purchasing items.

Sprint Planning

- When Sprint Goal is agreed upon team delivers the items into a “Done” product Increment
- Sprint planning isn't always completed in one meeting
 - Detailed plan for the first few days is enough
 - Development Team can prepare detailed plans for the rest of the work later
- Detail plan is a breakdown of a Product Backlog item into detailed tasks. Each task might have estimates, dependencies, and similar information for tracking.

Sprint Planning

- Sprint Goal (To Do, Doing, Done)
- The goal of sprint is:
 - To make the purchasing part of the website mature enough to be able to handle the whole process and users can experience a full purchasing process, through which other functionalities of the website will be more meaningful.
- Post-its are tasks created by breaking down each user story
 - Tasks define what the Development Team will do
 - Whole team is responsible for preparing them
 - Some tasks are created at the Sprint Planning meeting, and others throughout the Sprint

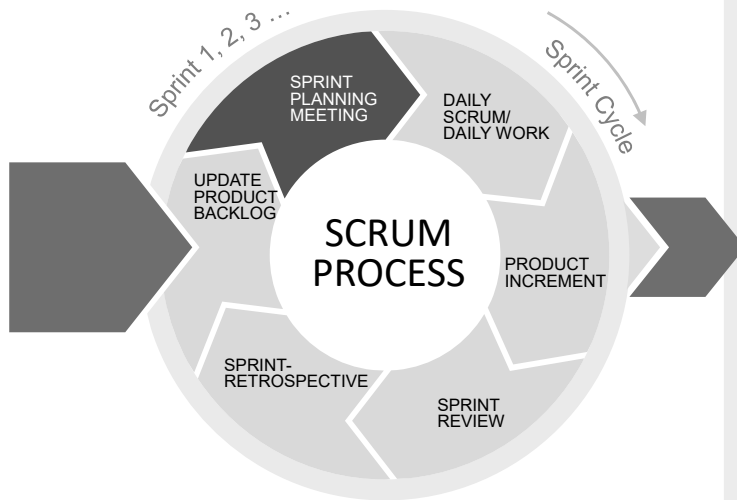
Sprint Planning

- The Sprint Backlog consists of:
 - The Sprint Goal
 - Selected items from the Product Backlog
 - Detailed plan for turning stories into “Done” Increment

Quick Scrum Review

End of lecture

AGILE PROCESS OVERVIEW



SPRINT PLANNING MEETING

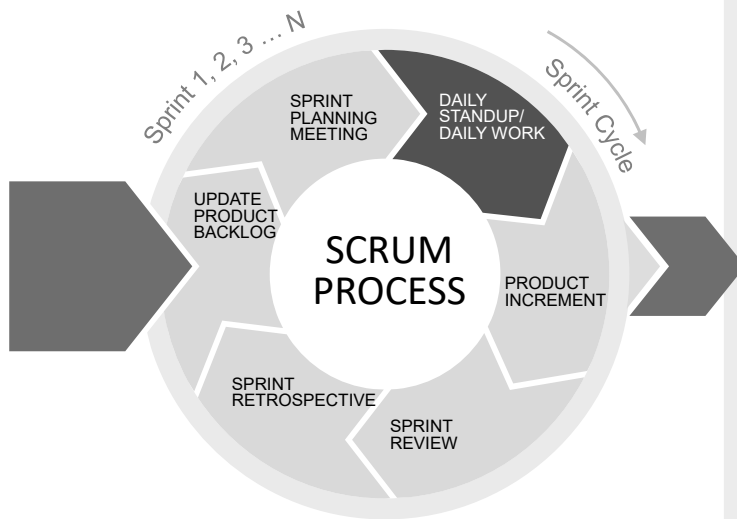
Sprint Planning Meeting plans the work to be performed.

Collaborative effort of the entire Scrum team leads to a plan.

What can be delivered in the Increment resulting from the upcoming Sprint?

How will the work required be achieved?

AGILE PROCESS OVERVIEW



STANDUP MEETING and DAILY WORK

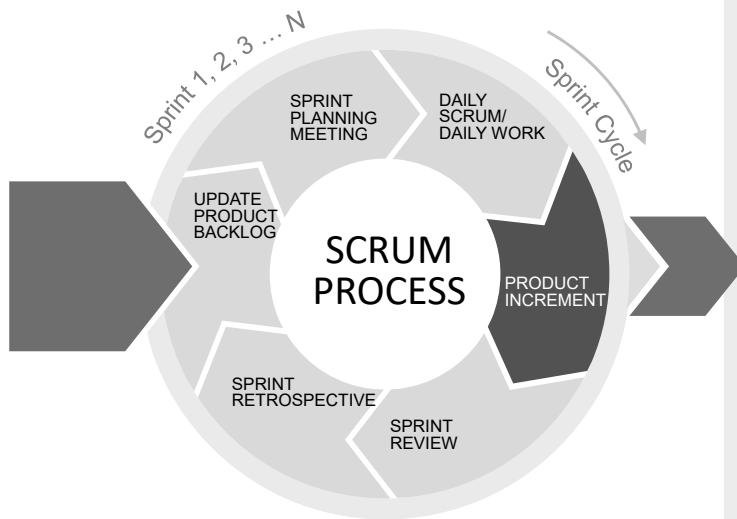
15-minute event

Development Team synchronizes activities and create a daily plan

Three questions:

1. What did I do yesterday that helped the Development Team meet the Sprint Goal?
2. What will I do today to help the Development Team meet the Sprint Goal?
3. Do I see any stumbling blocks that prevent me or the team from meeting the Sprint Goal?

AGILE PROCESS OVERVIEW



PRODUCT INCREMENT / FINISHED WORK

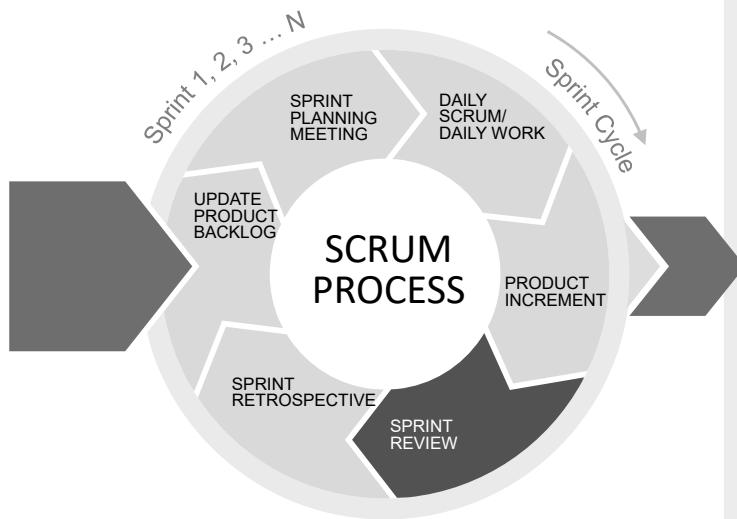
Increment is the sum of all the Product Backlog items completed during a Sprint

Includes the Increments of all previous Sprints.

At the end of a Sprint, the new Increment must be "Done".

The product must be useable regardless of whether the Product Owner decides to release it.

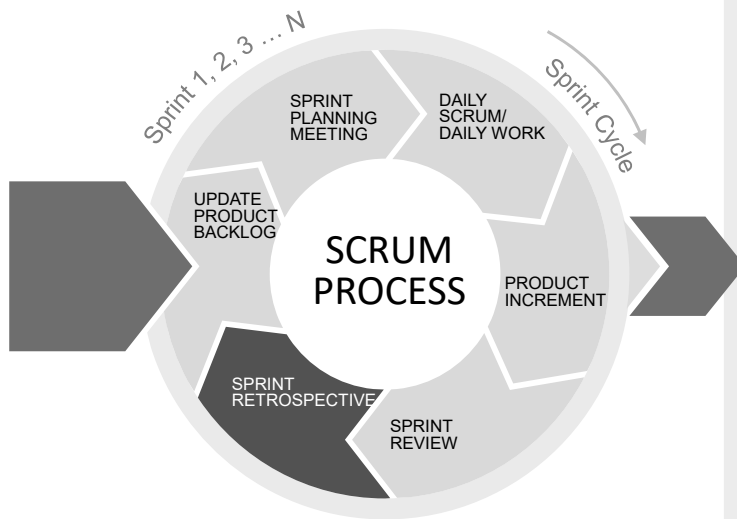
AGILE PROCESS OVERVIEW



SPRINT REVIEW

Sprint Review at the end of Sprint.
Inspect the Increment and may adapt the Product Backlog.
Scrum team and stakeholders collaborate about what was done in the Sprint.
Informal meeting, not a status meeting.

AGILE PROCESS OVERVIEW



SPRINT RETROSPECTIVE

Opportunity for the Scrum team to inspect and create a plan for improvement in the next Sprint. Occurs after the Sprint Review and prior to the next Sprint Planning. Opportunity to improve people, relationships, process and tools.

Product Backlog Grooming

- Reviewing and revising Product Backlog items
 - Adding detail
 - Creating estimates
 - Ordering items
- Product Owner is responsible for prioritizing
- Development Team is responsible for estimating

Product Backlog Grooming

- Main difference between grooming and the five Scrum events:
 - Scrum events are time-boxed
 - Grooming is an ongoing activity
- Grooming should not consume more than 10 percent of the Development team's time

Hosting a Daily Scrum

End of lecture

Daily Scrum

- 15-minute meeting for the Development Team to inspect the work since the last meeting
- Synchronize work and plan for the next 24 hours
- Three questions:
 - What has been accomplished since the last meeting?
 - What will be done before the next meeting?
 - What obstacles are in the way?

Daily Scrum

- Team assess progress towards Sprint Goal
- Forecasts likelihood of completing items before the Sprint is over
- Held at the same time and place throughout the Sprint
- For the Development Team; not a status meeting for all stakeholders
- Good idea for the Sprint board (wall chart) to be visible
- Burndown chart can be used to track remaining work

Working With Large Scrum Teams

- Scrum of scrums
- Scrum of scrums of scrums
- Four questions are answered:
 - What has your team done since we last met?
 - What will your team do before our next meeting?
 - Are there any roadblocks in your team's way?
 - Will your team put anything in another team's way?

Completing the Sprint Review

End of lecture

Sprint Review

- Four hours for a one-month Sprint
- Scrum Team and other stakeholders
- Present and inspect the “Done” items
- Collect feedback and change requests

Sprint Review

- Development Team does not present an item, unless it is 100%
- Product Owner makes sure (before the Scrum Review) that presented items are “Done”
- Development Team demonstrates and explains the items.
- Product Owner discusses the status of the Product Backlog and the likely completion dates
- Whole Scrum Team collaborates on Product Backlog

Performing a Sprint Retrospective

End of lecture

Sprint Retrospective

- Three hours for a one-month Sprint
- After the Sprint Review, and before the end of the Sprint, another meeting will be held, aimed at process improvement
- This is learning lessons and is the Sprint Retrospective
- Usually required, but optional if Product Owner, Scrum Master, and Development Team agree

Sprint Retrospective

- Always look for ways to improve
- Does not matter how little there should be an improvement
- Formal opportunity for improvement
- Participants reviews (inspects) the Sprint
 - People
 - Relationships
 - Processes and tools
 - Identify ways of improvement

Sprint Activity Considerations

End of lecture

Sprint Activities

- Sprint Planning meetings plan what will go into a Sprint
- The Product Owner prioritizes requirements and decides contents of the Sprint Backlog
- Stories make up Sprint Backlog
- Team breakdown stories into tasks

Sprint Activities

- Team takes 30 days or so to deliver an agreed amount of stories
- Daily Scrum of 15 minutes for team to collaborate with each other
- Sprint review team demonstrates the completed stories to customer in a Sprint Demo
- Scrum Retrospective team reviews Sprint and looks for improvement (lessons learned)
- Scrum Master makes sure the Scrum process is followed entirely and offers coaching

Sprint Duration Considerations

- Risk of being disconnected from stakeholders
- Ability to go to market with a product release
- Frequency that team composition can be changed
- All Sprints should be of same duration
- No such thing as Sprint Zero

Scrum Team Rules and Characteristics

End of lecture

Scrum Team

- Only three roles in a Scrum project
 - Additional roles harmful to team unity
 - Not compatible with Scrum philosophy
- “Scrum Team” refers to all the project team members
 - Product Owner
 - Scrum Master
 - Development Team members

It is possible for a single person to be assigned to more than one of the standard roles, but it is not recommended.

Scrum Team

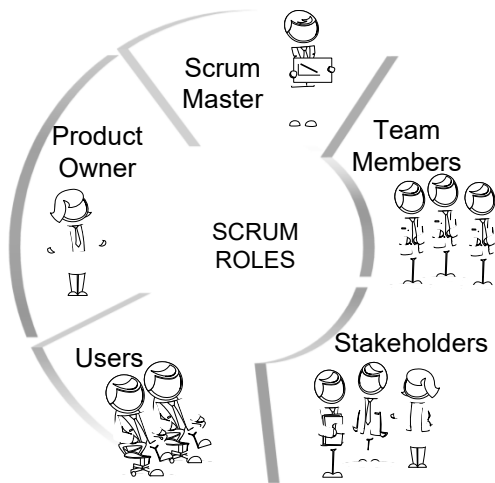
- Stakeholders can be involved in project, but are not considered internal to the team
- When the project is not internal, the customer is a stakeholder
- Always have external stakeholders

It is possible for a single person to be assigned to more than one of the standard roles, but it is not recommended.

Scrum Team Characteristics

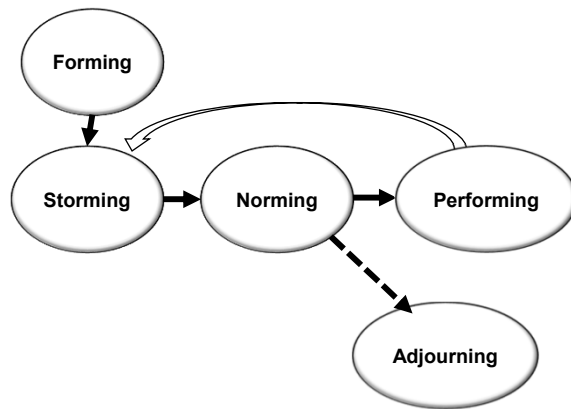
- Self-organized: Scrum Team manages own efforts
 - Not managed or directed by others
 - Management and specialist efforts are not separated in Scrum
- Cross-functional: Scrum Team has expertise and competencies
 - Get the job done without help from outside the team
- If work is too large for current team:
 - Remove or change selected items
 - Recruit additional Development Team members before the work begins

Scrum Stakeholders



Team Formation and Development

- Bruce Tuckman model



Agile Teams

- Most effective agile teams tend to range in size from:
 - Three to nine members
 - Located in a team space
 - 100 percent dedicated to the team

Agile Teams

- Self-managing teams
 - Members decide who will perform the what work in each iteration
- Agile teams thrive with servant leadership
 - Servant leadership – carry food and water mentality

Creating Collaborative Team Spaces

- Characteristics and location of the delivery team
- Work environment affects collaboration and communication
- Co-located teams are normal, but comes with challenges

Co-located Teams

- Not enough to locate all team members in the same city or same floor
- All members are working within 33 feet of each other
- No physical barriers

Osmotic Communication

- A benefit of co-located teams is osmotic communication.
- Useful information that flows between team members who are in proximity

Distributed Teams

- At least one team member is working off-site
- Majority of agile teams are distributed
- Communication tools and reduced communication costs
- Cost-effective approach

Global, Cultural, and Team Diversity

- Not uncommon to have team member from different countries working together.
- However, this may create challenges:
 - Time zones
 - Cultures
 - Communication styles
 - Native languages

Digital Tools for Distributed Teams

- Videoconferencing, live chat, Skype
- Interactive whiteboards
- Instant messaging (IM), and VoIP
- (Voice over Internet Protocol) headsets

Digital Tools for Distributed Teams

- Presence-based applications
- Electronic task boards and story boards
- Web-based meeting facilitators
- Survey applications
- Agile project management software
- Virtual card walls

Distributed vs. Outsourced Teams

- Distributed team: multiple development sites that can span buildings, cities, or countries
- Outsourced team: multiple legal entities, contracting, contract administration and different development infrastructures

Encourage Emergent Leadership

- Team success is more important than individual success
- Anyone can be a leader on an agile team
- Understanding roles and responsibilities

Fostering Emergent Leadership

- Allow team members to take charge
- Motivate the team members
- Show recognition of individuals and team
- Avoid zero sum rewards

Meet the Product Owner

End of lecture

Product Owner

- Work with stakeholders, customers, and teams to define direction
- Role belongs to one person
 - Can be a committee, but should be one person representing committee
 - Do not need to be developers
 - Well-versed in how the business operates

Product Owner

- One Product Owner for entire project
- One Product Backlog for entire project

Product Owners

- Guide the direction of the product
- Rank the work based on its business value
- Provide feedback
- Set direction on next deliverable

Product Owner

- Product Owner is responsible for the Product Backlog
 - Ensures each user story is easy to understand
 - Communicates with customers to keep the Product Backlog updated
 - Measures the performance of the project
 - Forecasts completion date and makes this information transparent

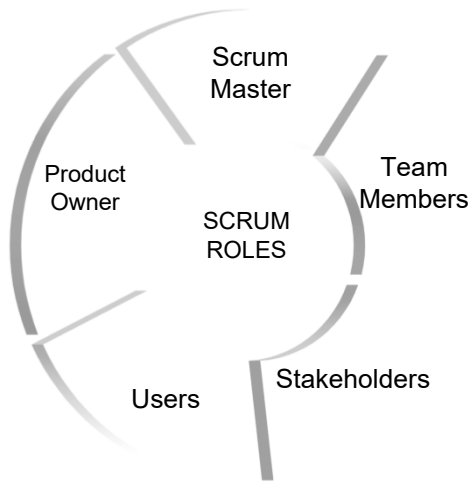
Product Owner

- Team and Product Owner work together:
 - Too much work a Sprint
 - Product owner can cancel a Sprint, not the Development Team

Product Owner

- Entire organization must respect the Product Owner decisions
- No one, even the CEO, should try to override decisions
- No one should tell the Development Team what item to deliver, except for the Product Owner
- Product Owner might delegate some responsibilities to the Development Team, but stays accountable for them

Product Owner and Customers



PRODUCT OWNER

- One person
- Owns product backlog
- Can cancel a sprint (Rare)
- Represents business
- Value optimizer



You, the Scrum Master

End of lecture

Scrum Master

- Scrum Master fully understands Scrum
- Coaches the Scrum team to ensure all Scrum processes are implemented (eg, Daily Scrum)
- Management position, which manages the Scrum process, rather than the Scrum Team
- Servant-leader for the Scrum Team
- Leads the organization in its effort to adopt Scrum

Scrum Master

- Removes impediments to the Development Team, facilitates events, and trains and coaches team
- Helps Product Owner by consulting on finding techniques, communicating information, and facilitating related events
- Helps those outside the Scrum Team understand the appropriate interactions with the Scrum Team
- Possible for a single person to be Scrum Master and a team member, although this is not recommended

Scrum Master

- Acts as a shield for team
- Can remove team members that are causing conflicts
- Keep stakeholders abiding by rules
- For example, only inspecting an increment at the Sprint Review

Scrum Master and Impediments

- When there are serious or many impediments:
 - Alert management to the impediments and impact
 - Consult with the Development Team
 - Prioritizing the impediments list and address each in order

Scrum Master and Product Owner

- Scrum Master is a servant leader
- Product Owner engages stakeholders
- Product Owner must be available to the Development Team:
 - Scrum Master can Inform the Product Owner's functional manager
 - Scrum master can address the problem in the Sprint Retrospective

Development Team

End of lecture

Development Team

- Experts that are responsible for delivering backlog items, and managing their efforts
 - Cross-functional: capable of the creation of each Product Backlog item
 - Self-organized: find their own way instead of receiving orders
- Whole Development Team responsible and accountable

Development Team

- No individual owns any task
- Development Team delivers the final product of the project in step by step Increments, as defined in the Product Backlog

Development Team

- Works full-time in a single project
- Development Team members should not change often
 - Team member changes should not happen during a Sprint
 - There will be a short-term decrease in productivity
- Scrum is effective when there are 3-9 Development Team members

Self-Organizing Development Team

- Boost to creativity
- Team and project commitment
- Accuracy of estimates
- When new teams are starting a project:
 - Ensure the team understands they need a definition of done
 - Scrum Team members introduce themselves and give a brief background of their skills and work history
 - Product Owner discusses the product or project, its history, goals, and context, as well as answer questions

Development Team Considerations

- Bring the team together and let them self-organize
- Existing teams can propose how they want to organize
- Large projects use a scaled model with multiple Scrum Teams
 - Multiple Scrum teams are not common
 - Adding new Scrum teams won't affect current productivity
 - Multiple Dev Teams should be self forming based on vision and Scrum rules
 - All teams need a common definition of done
 - All teams should have same Sprint starting date

Development Team and DoD

- Development Team should deliver additional features in a useable state that complement those delivered in previous iterations
- If the team can't finish all Sprint Backlog items:
 - They do not include the items in the increment of current Sprint
 - They do not show it in the Sprint Review
 - They must estimate it and return it to the Product Backlog for the Product Owner to decide what to do with it the item(s)

Considering Other Agile Roles...

End of lecture

Other roles

- Scrum does not allow this!
- Members have the same role and title: Development Team member
- Different titles or roles shifts focus to specific role and individuals might not pay enough attention to the final product
- Development Team members are responsible for all the outputs created in the Development Team

Who's the project manager?

- No such role in Scrum
- None of the roles act as a traditional project manager
- Scrum Master responsibilities are different than traditional PM

What happens to project management?

- PM responsibilities are distributed among the three roles
- There is no centralized project management in Scrum

Management and Leadership

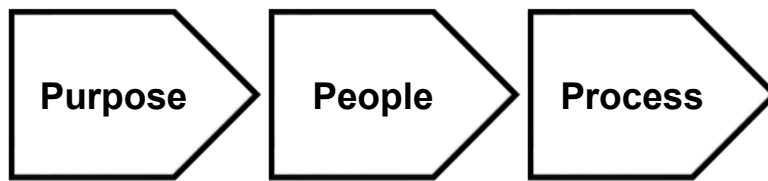
End of lecture

Management vs. Leadership

- Management: getting things done
- Leadership: getting people to want to do what needs to be done
- Management is more concerned with task control command

Servant Leadership Empowers the Team

- Servant leadership is the practice of leading through service to the team, by focusing on addressing the needs of the team to enable the highest possible team performance.



Servant Leadership Characteristics

- Promoting self-awareness
- Listening
- Serving those on the team
- Helping people grow
- Coaching versus controlling
- Promoting safety, respect, and trust
- Promoting the energy and intelligence of others

Servant Leader Responsibilities

- Manage relationships to build communication and coordination
- Educate stakeholders around why and how to be agile
- Support the team through mentoring, encouragement
- Celebrate with the team and external groups
- Help team with technical project management activities

Servant Leaders Facilitate

- Servant leaders are responsible to streamline processes that are impeding a team or organization
- Servant leaders shift the emphasis shifts from managing coordination to facilitating collaboration

Primary Role of Servant Leader

- Shield the team from interruptions
- Remove impediments to progress
- Communicate and re-communicate the project vision
- Carry food and water

Leadership Tasks

- Give transparency through visualization
- Create a safe environment for experimentation
- Experiment with new techniques and processes
- Share knowledge through collaboration

Considerations of Agile Projects

End of lecture

Architecture and Infrastructure Concerns

- Added to the Product Backlog and addressed in early Sprints
- Implemented along with functional development of the product
- Security, uptime, and non-functional requirements should be added to Product Backlog and DoD

Time-boxed Concepts

- Time-box is a fixed period
- Freeze the target and work with full focus on certain tasks or objectives
- Duration of a time-box should be agreed upon and fixed
- Free to change the duration based on lessons learned, but not frequently, and never based on single occasions

For example, we are not allowed to say that “we have a lot to do this time, so let's increase the duration for this particular case”. What we are allowed to say is “based on the previous ten time-boxes, we realized that the duration of our time-boxes is not suitable, and a 30% increase in duration might better fit our needs. So, let's increase them from now on”.

Pre-Sprint Activities

- Vision Statement: concise description of the goals of the project
- Product Roadmap: visual timeline of major product features to be delivered and is normally created by the Product Owner
- Stories: requirements normally written by the Product Owner and come from customer requirements
- Product Backlog: made up of stories and prioritized

Sprint Activities

- Sprint Planning meetings plan what will go into a Sprint
- The Product Owner prioritizes requirements and decides contents of the Sprint Backlog
- Stories make up Sprint Backlog
- Team breakdown stories into tasks

Sprint Activities

- Team takes 30 days or so to deliver an agreed amount of stories
- Daily Scrum of 15 minutes for team to collaborate with each other
- Sprint review team demonstrates the completed stories to customer in a Sprint Demo

Sprint Activities

- Scrum Retrospective team reviews Sprint and looks for improvement (lessons learned)
- Scrum Master makes sure the Scrum process is followed entirely and offers coaching

Sprint Duration Considerations

- Risk of being disconnected from stakeholders
- Ability to go to market with a product release
- Frequency that team composition can be changed
- All Sprints should be of same duration
- No such thing as Sprint Zero

Agile planning

End of lecture

Beginning with Adaptive Planning

- Agile projects are value-driven
- Minimize non-value-added work
- Plan to replan
- Early plans are necessary, but they're likely flawed
- Uncertainty requires replanning

Agile Versus Non-agile Planning

- Trial and demonstration uncover true requirements
- Less up front and more iterative planning
- Mid-course adjustments are normal

Trial and Demonstration

- Prototypes help initial planning
- Helps to avoid the gulf of evaluation
- Communicate agile planning practices

Iterative Planning

- The planning effort is distributed throughout the project life cycle
- Agile projects are risky
- Consider planning efforts over the project life cycle
- Agile projects typically do more planning

Principles of Agile Planning

- Plan at multiple levels
- Engage the team and customer in planning
- Demonstrate progress in velocity
- Tailor processes for the project

Principles of Agile Planning

- Priorities will cause the plan to be updated
- Utilize estimate ranges
- Base projections on completion rate
- Factor in diversions and outside work

Agile Goal: Deliver Value

Deliver Value Early in the Project

- Value is delivered first
- Longer projects mean more risk
- Delivering value early means team understands customers' needs
- Early value help stakeholders maintain synergy and interest

Making Value-Based Decisions

- What adds the most value?
- What is the definition of value for the project?
- Think of the customer of the project
- Value is in working software
- Value is in results

Minimize Waste is a Constant Goal

- Waste reduces value
- Poppendieck's Seven Areas of Waste:
 - Partially done work
 - Extra processes
 - Extra features
 - Task switching
 - Waiting
 - Motion
 - Defects

Identify Waste

- What's lurking to do in the project?
- Where are bottlenecks and time sucks?
- Are team members pulled between activities?
- What are you and the team waiting for?
- What type of escaped defects have you experienced?

Slack in Scrum

End of lecture

Slack in Scrum

- Does not matter how much the Development team works
- What is produced is what's important
- Team should be product-oriented, not activity-oriented
 - Limit the work time to a reasonable amount, and have frequent off times
 - Recommended to have a slack between each two Sprints
 - A day or two off to recharge batteries
 - Read some relevant articles
 - Check out what other teams are doing

Slack in Scrum

- Slacks can also be used for reading articles, taking part in courses or workshops, spending time on creative projects, etc.
- After the slack, repeat the same cycle until the final product of the project is delivered, and the client is completely satisfied
- Slack is not an event and the official Scrum.org does not mention it

Backlogs and Getting Done

End of lecture

Product Backlog

- Ordered list of everything that might be needed in the final product
- All items are described in simple business language
- All items estimated in story points
- Every requirement and every change in the project will be reflected in the Product Backlog
- Dynamically changing and improving
- Team does not wait until the Product Backlog is complete to start delivering the items

Product Backlog

- First Sprint can be started as soon as the Product Backlog has a sufficient amount of stories
- Product Owner sets a number of factors to determine the value of each item for the business
- Return on investment is one of the factors

Product backlog

- All factors will be summarized into one value
- Product Backlog ordered based on item value
- Higher-valued items be delivered sooner by the Development Team

Scrum Artifact – Product Backlog

- The product backlog is the source for all product requirements
- The product owner sorts and prioritizes the backlog items
- The development team always works on the most important items based on the prioritized items in the product backlog

Scrum Artifact – Product Backlog

- The backlog is always prioritized before the current sprint
- Backlog refinement is done by both the product owner and the development team working in harmony
- The team estimates their capacity to attack the items in the product backlog

Scrum Artifacts – Sprint Backlog

- Sprint backlog is a subset of the product backlog
- Sprint backlog serves as the goal for the current iteration
- Sprint backlog is a view into the work to be accomplished in the current sprint
- Sprint backlog is updated and refined by the development team

Sprint Backlog

- Created during the Sprint Planning which is the first event in a Sprint
- Sprint Backlog includes:
 - Number of items selected from the top of the Product Backlog
 - Sprint Goal to help describe the real meaning of the items and direct the efforts of the Development Team
 - Detailed plan for delivery of the items and realization of the Sprint Goal

Sprint Backlog

- Sprint Backlog is frozen after the Sprint Planning
- Development Team focuses on delivering an Increment of “Done”
- Stories in the Sprint Backlog cannot be added or removed
- Might be necessary to get more information, justify, or clear some of the items during the Sprint, which should be done in the presence of the Product Owner
- Detailed plan will become more complete as the Sprint continues

Definition of Done

- Shared understanding of what it means for a piece of work to be "Done"
- Definition of "Done" must be discussed and agreed upon at the beginning of the project so that future Increments would be releasable
- Over time, the team will improve their definition of "Done" to include more stringent criteria

Definition of Done

- Multiple Scrum Teams on a single project:
 - Might not be possible to use the same definition of “Done” for all teams, because they might be working on items of different natures
 - Each Scrum Team will define its own definition of “Done” and delivers its items based on that definition
 - Integration of definitions of “Done” should be capable of creating a potentially releasable Increment at the project level

Scrum Artifact – Product Increment

01

The product increment is the outcome of an iteration

02

The product increment is a chunk of the project work

03

The development team and the product owner must be in agreement of what done means for an increment

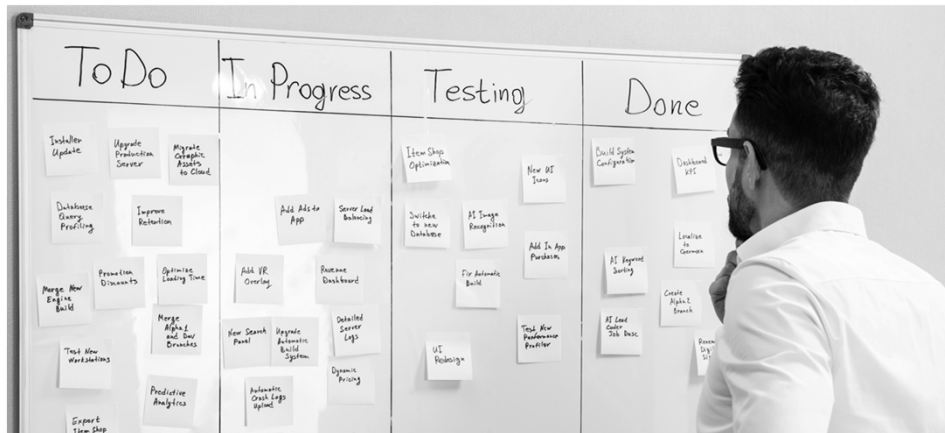
Monitoring Project Progress

End of lecture

Monitoring Project Progress

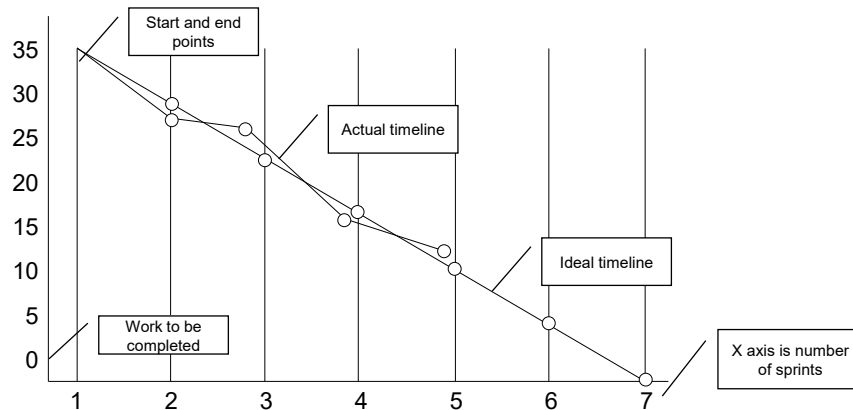
- Product Owner responsible to monitor the progress of the project
- Should be done at least once per Sprint Review
- Product Owner determines the amount of remaining work and compares it to the remaining work of the previous Sprints
- Forecasts the completion date of the project
- All stakeholders should have access to this information

Kanban Board



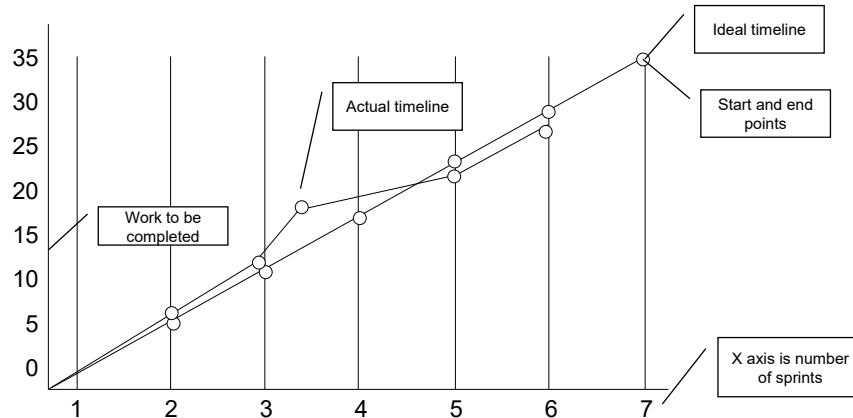
Utilizing Burndown Chart

- Track the work that remains to be done on a project
- Measures the team progress in completing the project work



Utilizing Burn Up Chart

- Track the work that has been completed
- As work is done the line moves upward



Understanding Team Velocity

- Velocity is the measure of a team's capacity for work per iteration
- Measured in the same unit that the team estimates the work
- Velocity very early and then stabilizes
- Velocity tends to plateau

Calculating Completion Time

- The team's velocity has been 20 story points per iteration
- There are 200 story points left
- Each iteration is two weeks
- $200 \div 20 = 10$
- $10 \times 2 = 20$
- There are 20 weeks left in the project

Monitoring Sprint Progress

- Monitor the progress of each Sprint throughout its life
- Responsibility of the Development Team
- Should be done in each Daily Scrum
- Used to calculate the likelihood of achieving the Sprint Goal and completing all items of the Sprint Backlog

Monitoring Sprint Progress

- Utilize a Sprint board for transparency
 - Sprint burndown chart
 - Sprint goal
 - Kanban of user stories

EVM

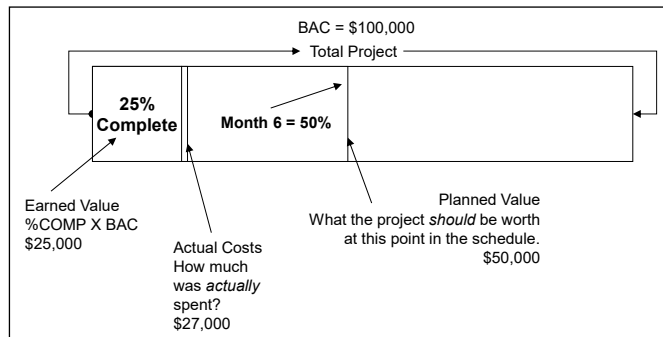
End of lecture

Earned Value Management in Agile

Earned Value Management For Agile Projects

- EVM is a suite of formulas to show performance
- Earned value compared to actual performance to planned performance

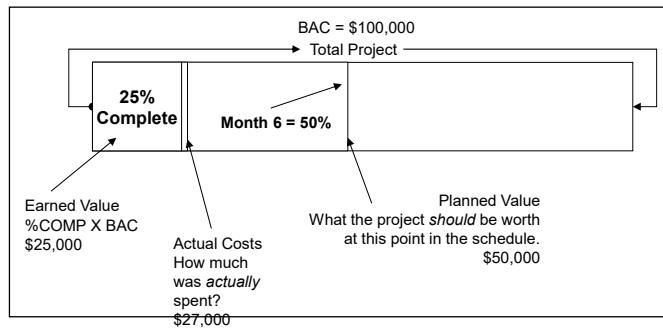
EVM Foundation



For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

Finding the Variances

- Cost variance= EV-AC
- Schedule variance=EV-PV

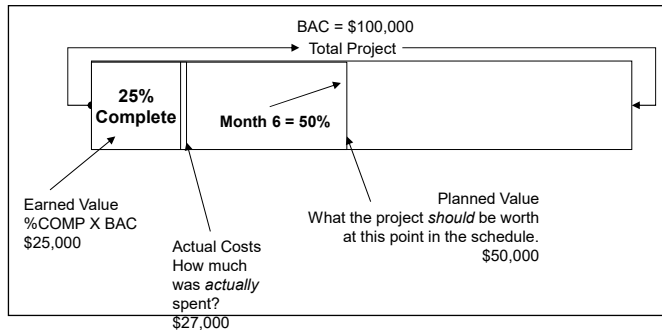


For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

Measuring Performance

CPI= EV/AC
CPI=25,000/27,000
CPI=.93

SPI=EV/PV
SPI=25,000/50,000
SPI=.50

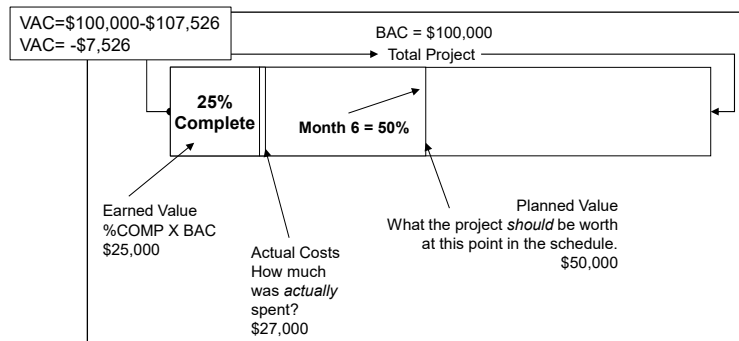


For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

Predicting the Future

- Estimate at Completion = BAC / CPI
- Estimate to Complete = $EAC - AC$

$EAC = \$100,000 / .93$
 $EAC = \$107,526$
 $ETC = \$107,526 - \$27,000$
 $ETC = \$80,526$



For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

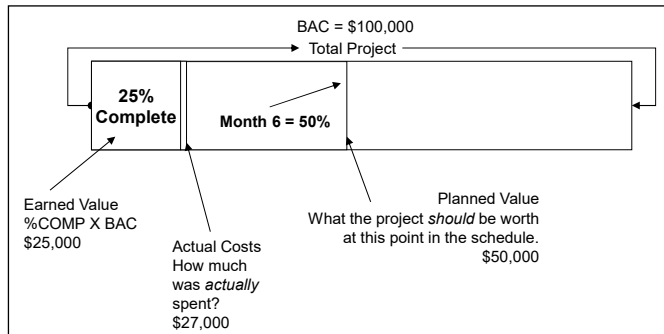
To-Complete Performance Index

- Efficiency needed to meet BAC:
 - $TCPI = (BAC - EV) / (BAC - AC)$
- Efficiency needed to meet EAC:
 - $TCPI = (BAC - EV) / (EAC - AC)$
- Great than 1, hard to accomplish
- Exactly 1, same level of efficiency
- Less than 1, easier to accomplish

For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

$$TCPI = (BAC - EV) / (BAC - AC)$$

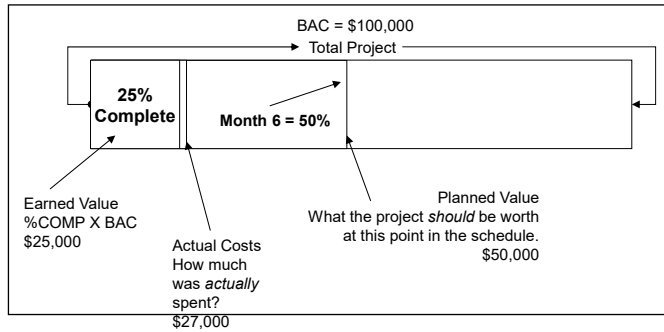
- $TCPI = (\$100,000 - \$25,000) / (\$100,000 - \$27,000)$
- $TCPI = 75,000 / 73,000$
- $TCPI = 1.0273$



For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

$$TCPI = (BAC - EV) / (EAC - AC)$$

- $TCPI = (\$100,000 - \$25,000) / (\$107,526 - \$27,000)$
- $TCPI = 75,000 / 80,526$
- $TCPI = .93$



For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

Story Points and EVM

- Can use Story Points instead of dollars
- Planned to complete 20 story points
- Completed 18 instead
- $SPI = EV/PV$
- $SPI = 18/20$
- $SPI = .90$

Five EVM Rules

- EV is first
- Variance means subtract
- Index means division
- Less than one is bad in an index
- Negative is bad in a variance

For more information on this topic, see the *PMBOK® Guide* 7.4.2.1.

Agile Project Accounting

Agile Project Accounting

- Agile accounting defines the economic models of agile projects
- Project work and smaller chunks of a larger project
- Smaller chunks of work are less risky
- Accountability of invested in relation to the ROI

Intermittent Releases and Value

- MVP – Minimal Viable Product
- Incremental funding
- Incremental ROI
- Business usage in “chunks”

Key Performance Indicators

- KPIs are metrics to show how well the project is performing
 - Rate of progress
 - Remaining work
 - Likely completion date
 - Likely cost remaining

KPIs and Project Life Cycles

- Predictive life cycle
 - Performance measurement baseline
 - Scope
 - Schedule
 - Cost
- Agile life cycle
 - All about getting things done
 - Likelihood of targets

Measurements in Agile Projects

Measurements in Agile Projects

- Transitioning to agile means using different measurements
- Agile projects deliver finished work on a regular basis
- Project teams can use such data for improved forecasts and decision making

Agile Teams Measure Results

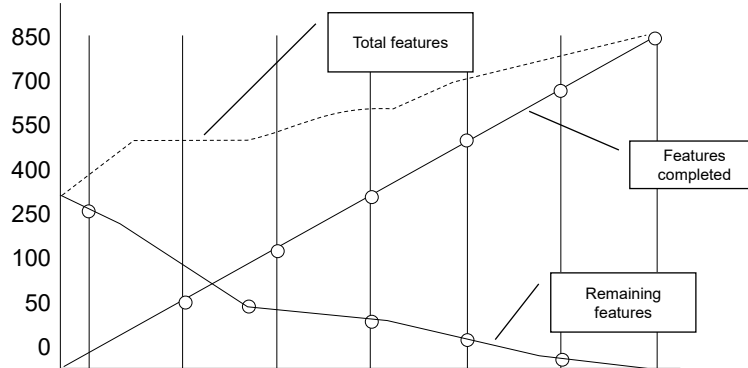
- Agile measures what the team delivers, not what the team predicts it will deliver.
- Agile teams limit their estimation to the next few weeks at the most. There is evidence that the smaller the chunk of work, the more likely people are to deliver.

Visual Display of Progress

- Burndown or burnup charts
- Completion as iteration progresses
- Information radiator

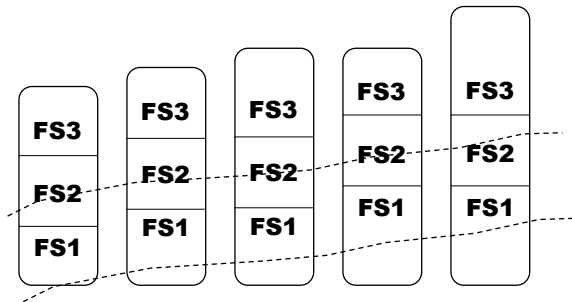
Features Chart

- The feature burnup/burndown charts may show that requirements grew during the project



Product Backlog Burnup Chart

- The team can show its completed value with a product backlog burnup chart as shown below.



Source: TOGETHER FOR THE FIRST TIME
PMBOK® GUIDE – Sixth Edition + Agile Practice Guide, Page 68

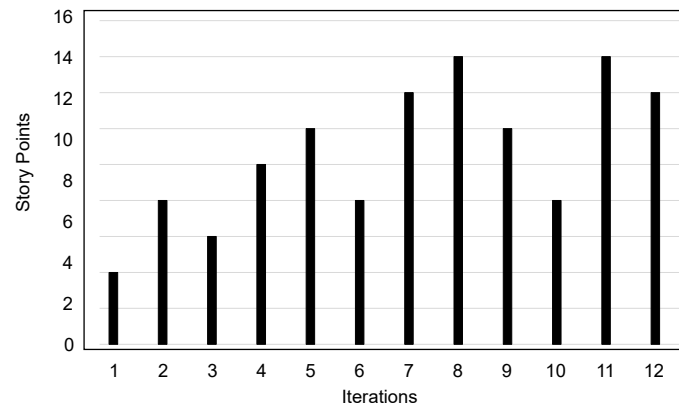
Understanding Team Velocity

Understanding Team Velocity

- Velocity is the measure of a team's capacity for work per iteration
- Measured in the same unit that the team estimates the work
- Velocity very early and then stabilizes
- Velocity tends to plateau

Measuring Team Velocity

- Measure of a team's capacity for work per iteration
- Team can gauge how much work can be accomplished based on the amount of work completed in past iterations.



Calculating Completion Time

- The team's velocity has been 20 story points per iteration
- There are 200 story points left
- Each iteration is two weeks
- $200 \div 20 = 10$
- $10 \times 2 = 20$
- There are 20 weeks left in the project

Progressive Elaboration

- As more information becomes available more planning can happen
- Continuing steadily in small increments
- Progressive elaboration examples
 - Plans
 - Estimates
 - Risk assessments
 - Requirements definition
 - Software design
 - Test scenarios

All About Kanban

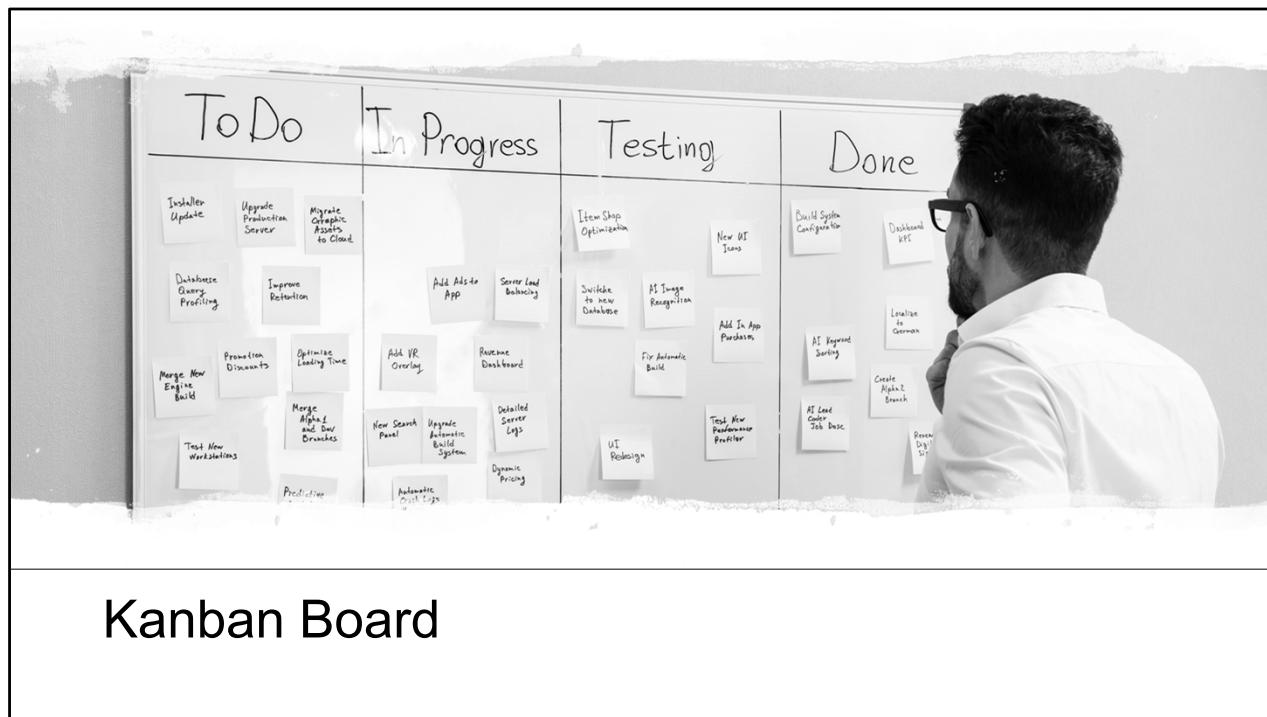
All About Kanban

- Originated at Toyota
- Japanese word meaning “signboard”

To Do	In Progress	Done
Create landing page	Server setup	Domain purchased
Build shopping cart	Domain routing	Logo created
Payment gateway	Printed materials	
Catalog search	Marketing emailer	
Site search		

Kanban Pull System

- A pull system moves work through development
- The development team completes an item; the next item in queue is pulled into the next stage of the process
- Kanban does not use time boxed iterations
- Only so many items can be in each stage of the project
- Work moves from left to right



Kanban Board

Principles of Kanban

- Visualize the workflow
- Limit work in progress (WIP)
- Manage the workflow
- Make process policies clear
- Aim for collaborative improvements

Kanban Pull System

- Work item completed
- Next item pulled into WIP
- Only so many items allowed in WIP
- Not a real need for iterations

Extreme Programming

Extreme Programming

- Extreme programming is also known as XP
- XP is all about software development best practices
- The exam will lightly test your knowledge of XP

XP Core Values

- Simplicity – reducing complexity, extra features, and waste
- Find the simplest thing that could possibly work

Extreme Programming Core Values

- Communication
 - Ensuring that the project team knows what is expected of them
 - Ensuring the project team knows what other people are working on
 - The daily standup meeting is an excellent communication tool

Extreme Programming Core Values

- Feedback:
 - The development team needs feedback early in the project
 - Failing fast is a way to get feedback early
 - Feedback gives the team an opportunity to improve the project

Extreme Programming Core Values

- Courage
 - Developers' work is entirely visible to others on the project team
 - Team members share code and correct each other's code
 - XP uses pair programming

Extreme Programming Core Values

- Respect
 - Team members must respect one another
 - Everyone is responsible for the success and or failure of the project
 - Everyone works differently but must work together

XP Team Roles

- Coach – mentor/guide/facilitator/communicator similar to the ScrumMaster
- Customer – the individual who provides requirements priorities and direction for the project similar to the product owner
- Programmer – the developers who write the code
- Testers – Define and write the acceptability test

XP Core Practices: Whole Team

- XP team members are collocated
- Generalizing specialist not role specialist
- Efficient and sharing of information

XP Core Practices: Planning Games

- Planning games are just planning activities
- Release planning is the release of new functionality
 - No more than one/two releases per year
 - The customer outlines the functionality required in the release
 - Developers estimate the difficulty to build the functionality

XP Core Practices: Planning Games

- Iteration planning is similar to sprint planning
 - Iteration planning happens at the start of every iteration
 - The customer defines what functionality they want to see by the end of the iteration
 - The development team estimates the difficulty to build the functionality

XP Core Practices: Small Releases

- Small releases to test environment are part of the XP practices
- Increases visibility to the customer
- Helps to deploy working software to the end users

XP Core Practices: Customer Tests

- Definition of the required functionality
- One or more test criteria for the software to be working

XP Core Practices: Collective Code Ownership

- Any pair of developers can improve or amend the code
- Multiple people will work on all the code
- Improve defect resolution in discovery
- Knowledge is shared not isolated

XP Core Practices: Code Standards

01

A coding standard
is defined

02

The team
adheres to the
standard

03

Provides for
consistency in
writing the code

XP Core Practices: Sustainable Pace

- Productivity is optimized through a sustainable pace
- Consistent overtime and long hours are not sustainable

XP Core Practices: Metaphor

- Metaphors and similes are used to explain designs
- Metaphors help communicate the software to the customer

XP Core Practices: Continuous Integration

- Compiling the code frequently throughout the day
- Programmers check-in code to the code repository
- Integration test run automatically for immediate feedback

XP Core Practices: Test-driven Development

Acceptance test
are written prior to
developing new
code



Initial tests will fail
because the code
has not been fully
developed yet



When the code
has been written
correctly it will
pass the test

XP Core Practices: Refactoring

- Cleaning up the code
- Removing duplicated code
- Lowering coupling
- Increasing cohesion

XP Core
Practices:
Simple
Design

What is the simplest thing
that could work?

Simple does not mean
easy

Simple design is a risk
mitigation approach

XP Core
Practices:
Pair
Programming

One person writes the code while
the second person reviews the
code

The two people change roles
frequently

The pair will catch mistakes and
speed up productivity

Lean Product Development

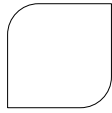
Lean Product Development

- Toyota production system
- Visual management tools
- Customer requirements to find value
- Learning and continuous Improvement

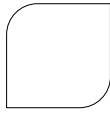
Seven Lean Core Concepts

- Eliminate waste
- Empower the team
- Deliver fast
- Optimize the whole
- Build quality in
- Defer decisions
- Amplify learning

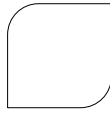
Seven Wastes Of Lean



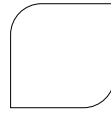
PARTIALLY
DONE WORK



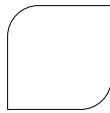
EXTRA
PROCESSES



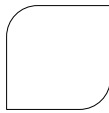
EXTRA
FEATURES



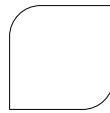
TASK
SWITCHING



WAITING



MOTION

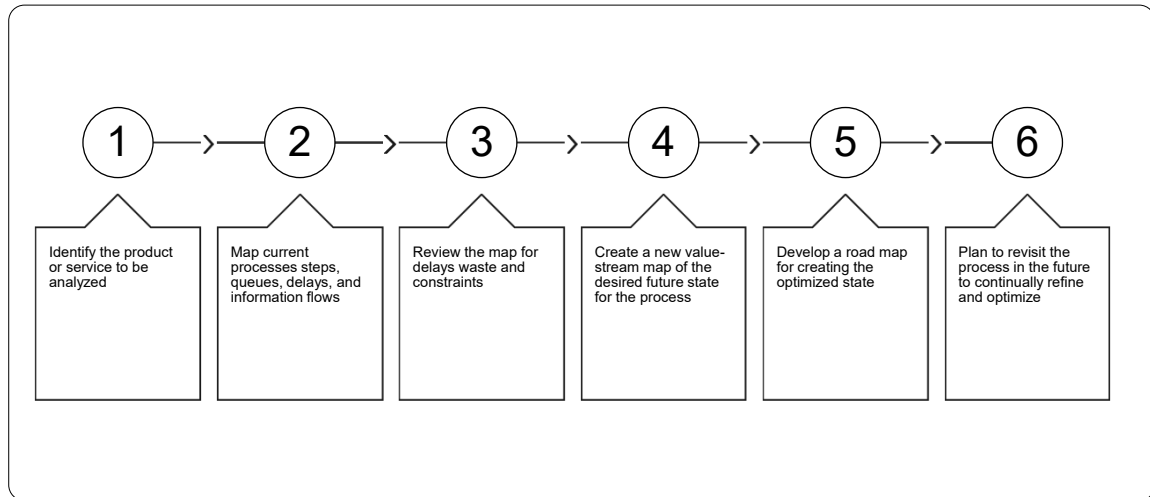


DEFECTS

Value Stream Mapping

- Lean manufacturing technique adopted by agile
- A visual map of a process flow to identify delays, waste, and constraints

How To Create a Visual Stream Mapping



Value Stream Mapping Example



Other Agile Approaches

Dynamic Systems Development Method

- Focus on the business need
- Deliver on time
- Collaborate
- Never compromise quality
- Build incrementally from foundations
- Develop iteratively
- Communicate continuously and clearly
- Demonstrate control

Feature-Driven Development

- The development team creates a model for the product
- They will build a feature list and a plan for the work
- The team moves through the design and build the directions for the product features
- The team designs by features and builds by features

Feature-Driven Development Characteristics

Domain
object
modeling

Developing
by feature

Individual
class code
ownership

Feature
teams

Inspections

Configuration
management

Regular
builds

Visibility of
progress and
results

Crystal

- Customized methodologies coded by color names
- Methodologies are appropriate for different criticalities and team sizes
- Criticality is about the impact of a product defect design

The Crystal methodology is one of the most lightweight, adaptable approaches to software development. Crystal is actually comprised of a family of agile methodologies such as Crystal Clear, Crystal Yellow, Crystal Orange and others, whose unique characteristics are driven by several factors such as team size, system criticality, and project priorities. This Crystal family addresses the realization that each project may require a slightly tailored set of policies, practices, and processes in order to meet the project 's unique characteristics. Several of the key tenets of Crystal include teamwork, communication, and simplicity, as well as reflection to frequently adjust and improve the process. Like other agile process methodologies, Crystal promotes early, frequent delivery of working software, high user involvement, adaptability, and the removal of bureaucracy or distractions. Alistair Cockburn, the originator of Crystal, has released a book, *Crystal Clear: A Human-Powered Methodology for Small Teams*.

Crystal Details

- Crystal Clear
- Crystal Yellow
- Crystal Orange and others
- Unique characteristics are driven by several factors such as team size, system criticality, and project priorities.
- This Crystal family addresses the realization that each project may require a slightly tailored set of policies, practices, and processes in order to meet the project 's unique characteristics.

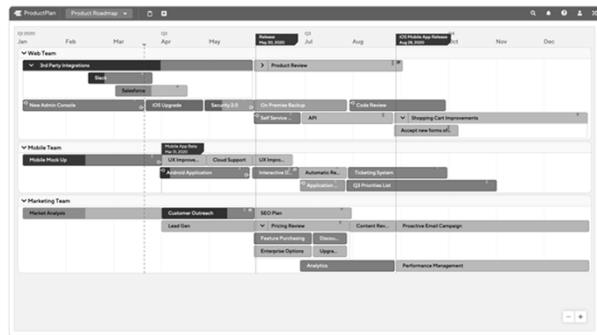
Releasing the Product

Product Vision

- A product vision is created
- The product must be in alignment with the company's strategy
- This is done by the product owner

Creating the Product Roadmap

- The visualization of product features
- The product roadmap equates to the product division as a whole
- This is done and owned by the product owner



Release Planning

- The release timing for specific product functionality
- Priorities are assigned to the product features from most important to least important
- These features become the product backlog
- Owned by the product owner

Iteration Planning

- Goals are established for the current Sprint or iteration
- Goals are based on the product backlog
- Product owner in the development team work together to prioritize features
- This is done at the start of each Sprint

Released Product

- The product is released according to the project's release plan
- Not every increment or completed iteration results in a release

Regulations, Risk, and Requirements

Regulatory Compliance for Agile Projects

- Regulations are requirements
- Regulatory compliance is one instance for documentation where just because is utilized

Managing Risk in an Agile Project

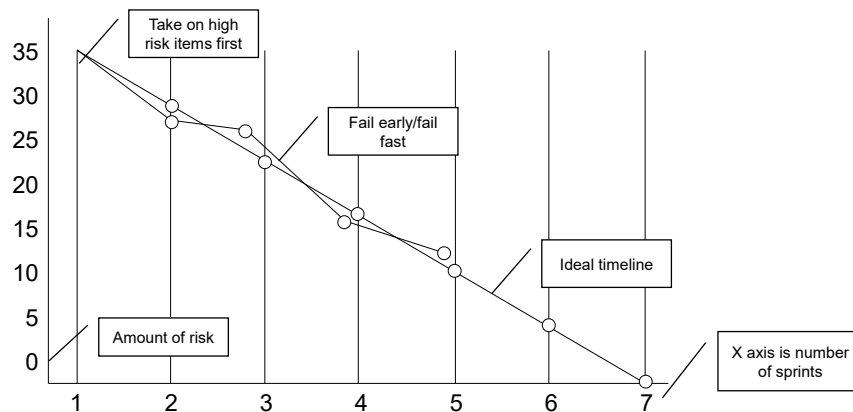
- Risk in an agile project is anything that threatens the project's goals
- Risk is considered an anti value
- Risk must be managed in a project
- Risk identification is an iterative activity
- Risk a recorded in a risk log

Addressing Features With High-risk

- Features with high risk addressed early in project iteration
- High risk need to be addressed sooner rather than later
- Risk-adjusted backlog brings risk to early portion of project
- Risk burndown chart tracks risk as they move down in priority and elimination

Risk Burndown Chart

Same concept as burndown chart for user stories



Creating Project Priorities

Customer Value Prioritization

Agile teams work on the items that yield the highest value to the customer first

The product owner is responsible for keeping items in the backlog prioritized by business value

When changes are added to backlog, they must be prioritized for value

The customer is the person who will declare what success looks like

The team will discuss with the customer at the end of each iteration the priority of the remaining work items

Prioritization Schemes

- How the work is prioritized
- The team agrees on the prioritization scheme
- The prioritization scheme is communicated and agreed upon by the entire agile team

Simple Scheme for Prioritization

- Items in the product backlog are ranked:
 - Priority one (high)
 - Priority two (medium)
 - Priority three (low)
- This approach has a risk that everything is ranked as priority one

Customer-Valued Prioritization

- What has highest value first?
- Grooming backlog for value
- Politics and organizational goals
- Product owner

MoSCoW Prioritization Scheme

- Made popular by DSDM
- Must have
- Should have
- Could have
- Would like to have, but not this time

Agile Project Prioritization Schemes

Minimum Viable Product

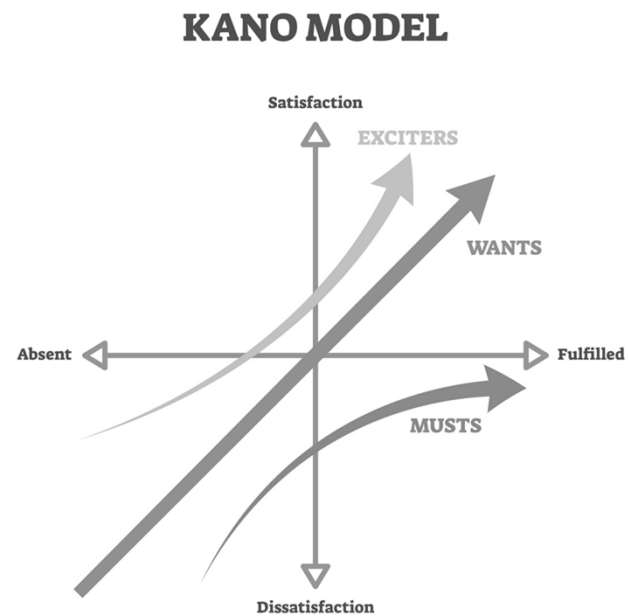
- Complete enough to be useful
- Small enough that it does not represent the entire project
- Also known as the minimal marketable feature
- Barebones essentials of a product

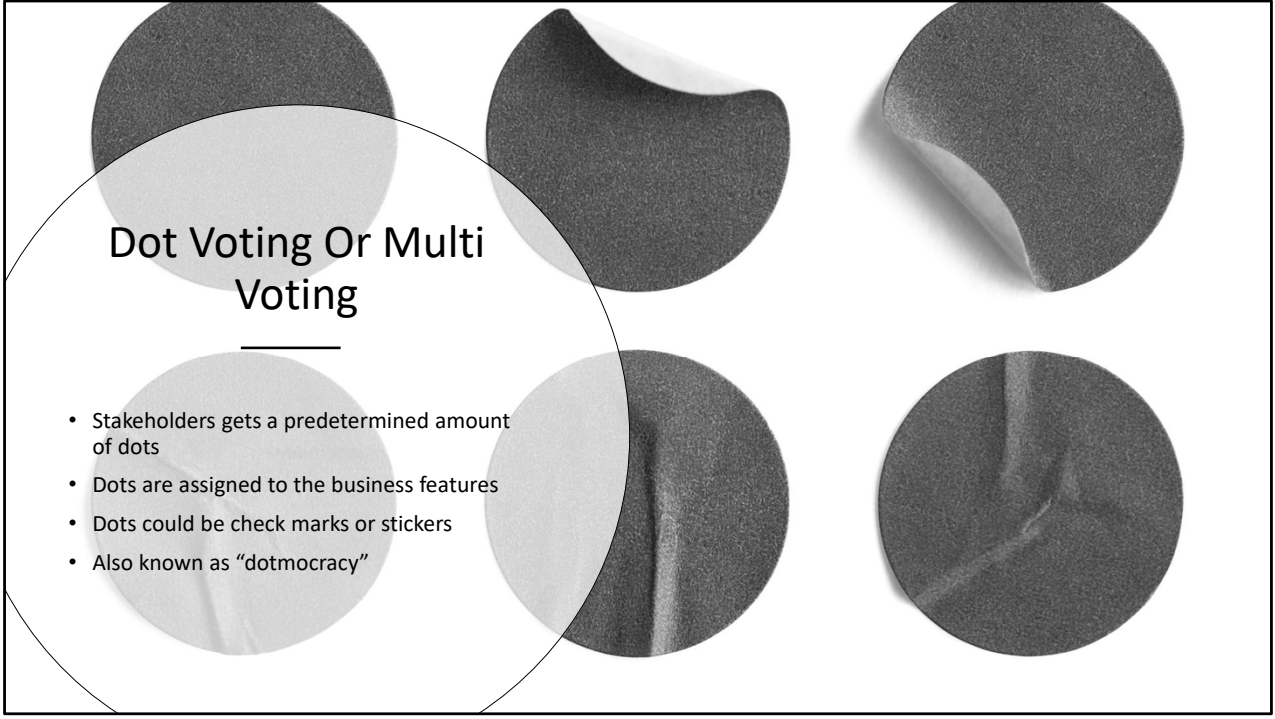
5 Categories of Customer Preferences

- Must-be quality
 - Ignored when met, dissatisfied when not met
- One-dimensional quality
 - Spoken attributes that are promises
- Attractive quality
 - Nice-to have features, but not necessarily required
- Indifferent quality
 - Customers are indifferent to the quality feature
- Reverse quality
 - Some customers will love, others will hate the feature

Kano Analysis

- Delighters excitors
- Satisfiers
- Dissatisfiers
- Indifferent





Dot Voting Or Multi Voting

- Stakeholders gets a predetermined amount of dots
- Dots are assigned to the business features
- Dots could be check marks or stickers
- Also known as “dotmocracy”

Dot Voting Variations

- Color of dots could represent opinion
 - Red or green
- Color of dots could represent stakeholder
 - Management
 - Team
 - Customer

Dot Voting Risks

- New features can't be considered once voting begins
- Review all features before voting – avoid too many options
- Similar options are basically penalized
- People follow crowd rather than consider options

Monopoly Money

- Stakeholders receive monopoly money equal to the amount of the project budget
- The monopoly money is distributed among the system features
- This approach is most effective when it's limited to prioritizing business features

Monopoly Money Considerations

- Consider cost of:
 - User-suggested features
 - Rival product features
 - Research-based features
 - Market research-based features

<http://www.uxforthemasses.com/buy-the-feature/>

100-Point Method

- Each stakeholder is allotted 100 points
- The points are assigned to the most important requirements
- Weighted distribution of points

100-Point Considerations

- Public or private voting
- Participants understand process
- Deceptively simple
- Vote on each feature

Requirements Prioritization Model

- Uses a scale of 1 to 9
- Benefit, penalty, cost, and risk of every feature

Considerations for Requirements Prioritization Model

- Value – business benefit
- Cost – expense versus ROI
- Risk – riskiest requirements first
- Difficulty of implementation – safest first
- Success factors – high probability of success
- Regulatory compliance – laws and regulations
- Relationship to other requirements – linked to other requirements
- Stakeholder agreement – consensus on requirement
- Urgency – time-sensitive requirement

Relative Prioritization Ranking

Priority of features

Simplify the features from most important to least important

Determination made to meet budget and schedule

Changes may change the prioritize list

Changes may bump some priorities from the list

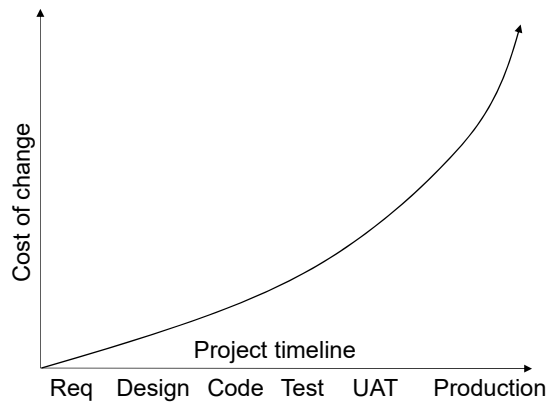
Prioritized Backlog

- List of all requirements
- Prioritized with product owner
- Priorities can change with each grooming
- Priorities don't change during the iteration

Reviewing the Cost of Change

- Change is least expensive at the start of the project
- Change increases as product functionality increases
- Change is expected, but can be expensive – and timely

Reviewing the Cost of Change



Low Tech and High Touch Solutions

Agile Tooling

- Agile teams prefer low-tech high-touch tools over-sophisticated computerized models
- Technical tools can exclude team members from interacting
- Consider high-tech tools for scheduling:
 - Data accuracy perception increases
 - A bad estimate is a bad estimate
 - Barriers for stakeholder interaction are created

Examples Of Low-tech High-touch Tools

- Cards
- Charts
- Information radiator
- Tools that promote communication and collaboration
- Tools that promote learning and knowledge transfer

Scheduling Software Vs. Kanban Board

- Also known as a task board
- Help teams monitor the work in progress



Digital Tools for Distributed Teams

- Electronic tools to mimic low-tech, high-touch tools:
 - Presence-based applications
 - Electronic task boards and story boards
 - Web-based meeting facilitators

Assignment: Creating a Sign Board

WIP and Little's Law

Work In Progress (WIP)

- Also known as a work in process and work in play
- WIP is risk
- WIP hides bottlenecks
- WIP requires investment
- WIP delivers no return until work is complete
- WIP needs to be limited

Limiting Work In Progress

- Agile attempts to limit WIP
- Kanban boards can have WIP limits
- WIP limits keeps the team from taking on too much work
- WIP limits reveal bottlenecks

Little's Law

- The duration of a queue is proportional to its size
- By limiting the work-in-progress teams complete work faster
- Lead time of a process is equal to the amount of work in progress divided by the average time it takes to complete each piece of work

$$\text{Average lead time} = \frac{\text{Average WIP}}{\text{Average Throughput}}$$

$$\text{Average lead time} = \frac{12}{56}$$

$$\text{Average lead time} = 4.6 \text{ days}$$

Working With User Stories

What is a User Story?

- Small chunk of business functionality within a feature that involves roughly 1-3 days work
- Also defined as for 40 hours of work
- User stories are written on index cards or sticky notes
- User stories are the items in the product backlog

Creating A User Story

- Potential stories are called candidate stories
- Perspective of the user or customer
- Often written in the following format
 - As a roll I want functionality so that business benefit
- Answers two questions:
 - Who was asking for this?
 - Why are we doing this?

User Story Formats

- Given – the scenario for the story
- When – the action that takes place
- Then – the result of the action

Three C's Of User Stories

- Card – just enough text to identify the story
- Conversation – details are communicated via a conversation between the customer and the development team
- Confirmation – customer confirmed the story has been implemented correctly

INVEST In User Stories

- Characteristics of effective user stories follow INVEST:
 - Independent – stories can be prioritized in any order
 - Negotiable – the team can discuss the user story with the product owner and make trade-offs based on the cost and function
 - Valuable – the user story must have obvious value
 - Estimate – the user story can be estimated for effort
 - Small – small user stories are easier to create and test than large user stories; 4 to 40 hours work
 - Testable – the story results must be testable

Hosting Workshops

Hosting Workshops

- Meetings for participants to get work done
- Clear goals and a schedule
- Retrospectives
- Planning meetings
- Estimating sessions

Tips for a Great Workshop

- Have a diverse group of people
- Facilitated for involvement
- Get people involved early

User Story Workshops

- Preferred approach for candidate user stories
- Also known as story writing workshops
- Optimize the workflow by understanding user needs
- Engage stakeholders in the design process

Brainstorming

- Collaborative technique too rapidly generate lots of ideas
- Maximize number of suggestions
- No stupid ideas
- Will sort through the ideas later

Brainstorming Methods

- Quiet writing
- Round robin
- Free for all

Collaboration Games

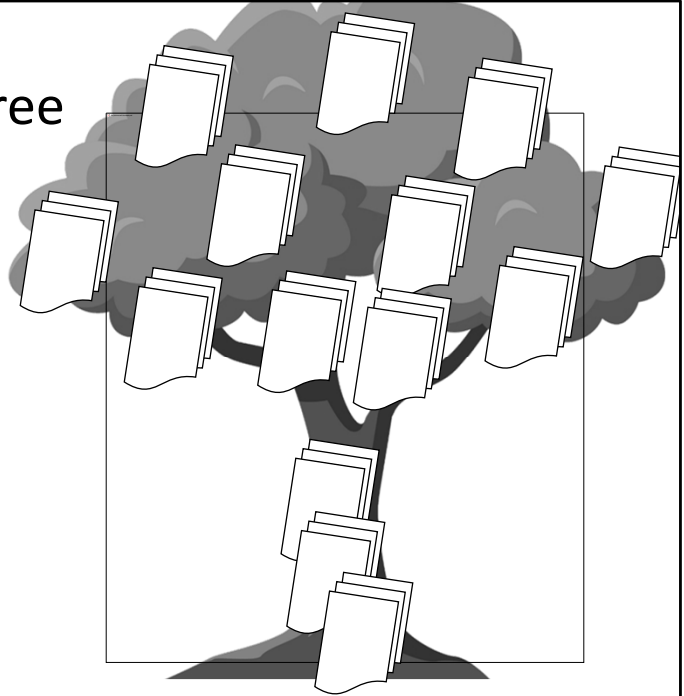
- Also known as innovation games
- Remember the future
- Prune the product tree
- Speed boat
- Buy a feature
- Bang for the buck

Remember the Future

- Collaboration game
- Stakeholders look back at the project
- 20 minutes to write a future report about how the project went
- Includes what was created; written on sticky notes
- Notes are moved into associated clusters and duplicates removed
- This game define success

Prune The Product Tree

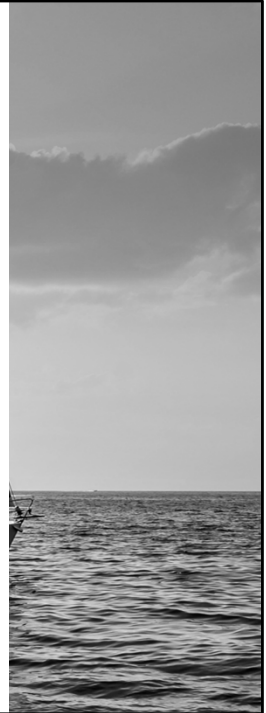
- Drawing of a big tree
- The trunk is what we already know or have built
- The branches are new functionality and what needs to be designed
- Participants add features on sticky notes to the tree
- Closer to the trunk represents higher priority





Speed Boat Games – Sailboat

- Imagine it's a boat
- What winds are pushing the sailboat
- What anchors are holding the sailboat back
- What direction is the sailboat going
- What rocks are in the way

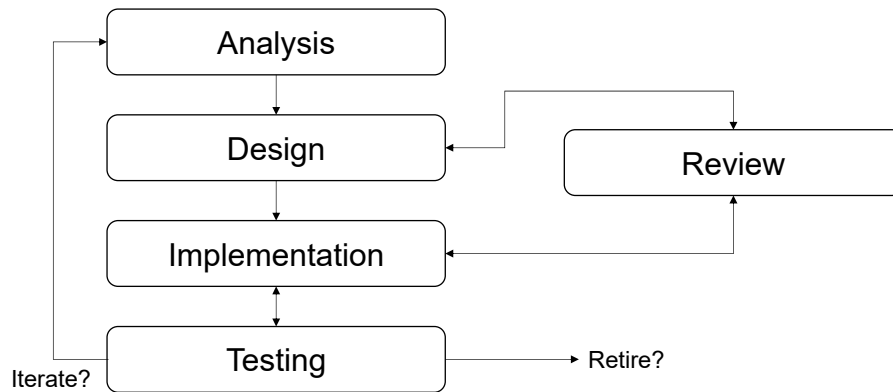


Working With Agile Modeling

Working With Agile Modeling

- Modeling techniques for agile projects
- The value is in discussion and creation of the model
- Often treated on whiteboards and photographs for a record
- Lightweight and barely sufficient

Agile Model Development



Agile Modeling Considerations

- Supplement to agile frameworks
 - XP
 - Scrum
- Adequate level of document (light, just enough)
- Difficult to do with large teams, non-collocated
- Face-to-face communication preferred

XP: Agile Modeling Values

- Communication – among team and project stakeholders
- Simplicity – easier to explore an idea by drawing a diagram instead of writing lines of code
- Feedback – communicating ideas through diagrams to quickly gain feedback
- Courage – make decisions and changes by discarding or refactoring
- Humility – humility to recognize that we don't know everything

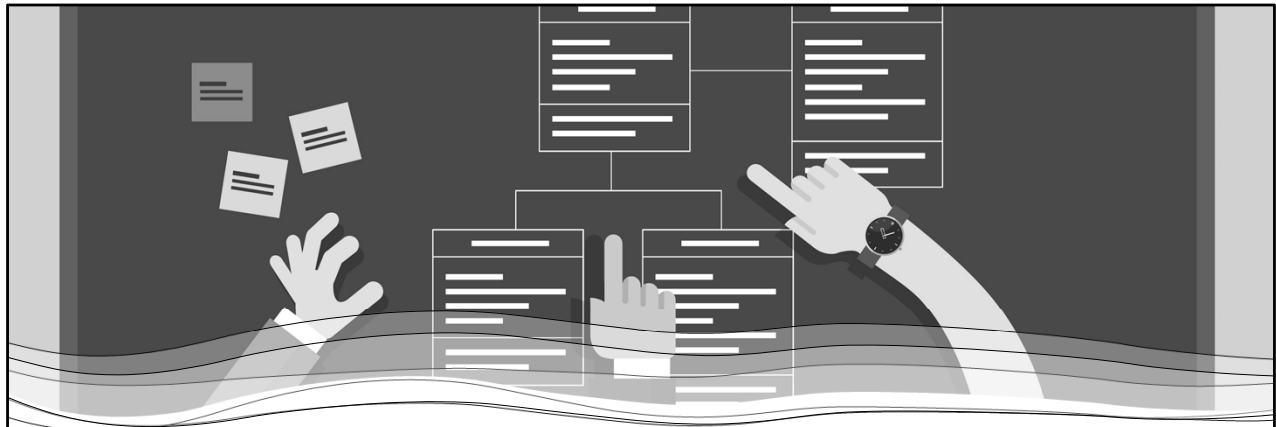
Use Case Diagram Purpose

Use Case Diagram Purpose

- Model the functionality of a system using actors and use cases
- Use cases are a set of actions, services, and functions
- Actors are people or entities operating within the system

Use Case Diagram Components

- System – a rectangle that contains use cases; actors are outside the boundaries
- Use Case – ovals; label with verbs that represent the functions
- Actors - the users of a system. One system could be an actor of another system
- Relationships – line; the relationships between an actor and a use case



How will the user use the solution?

Use Case Diagram

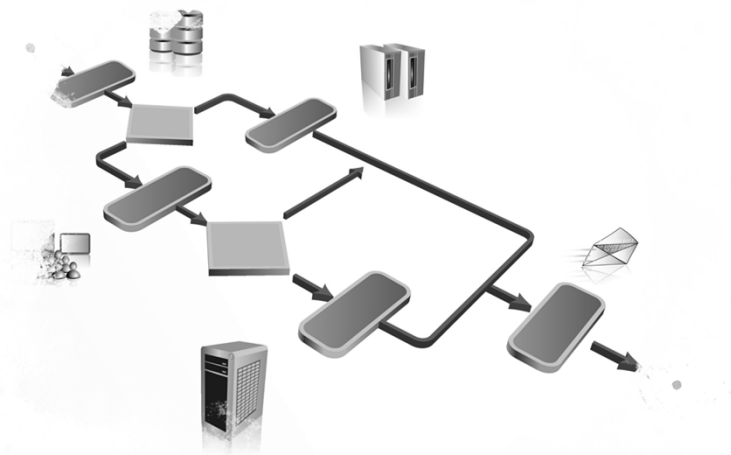
Defining Data Models

Defining Data Models

- Determines the structure of data
- Sometimes called a data structure
 - Car
 - Engine
 - Exhaust
 - Tires
 - Windows
 - Interior
 - Front
 - Back
 - Trunk

How data flows through the system

Sample Data Model



Considering Screen Designs

- User interaction
- User stories are product focused
- Promote cooperation between team and product owner
- Avoid creep, design black holes

Screen Designs

- What does the user interface look like

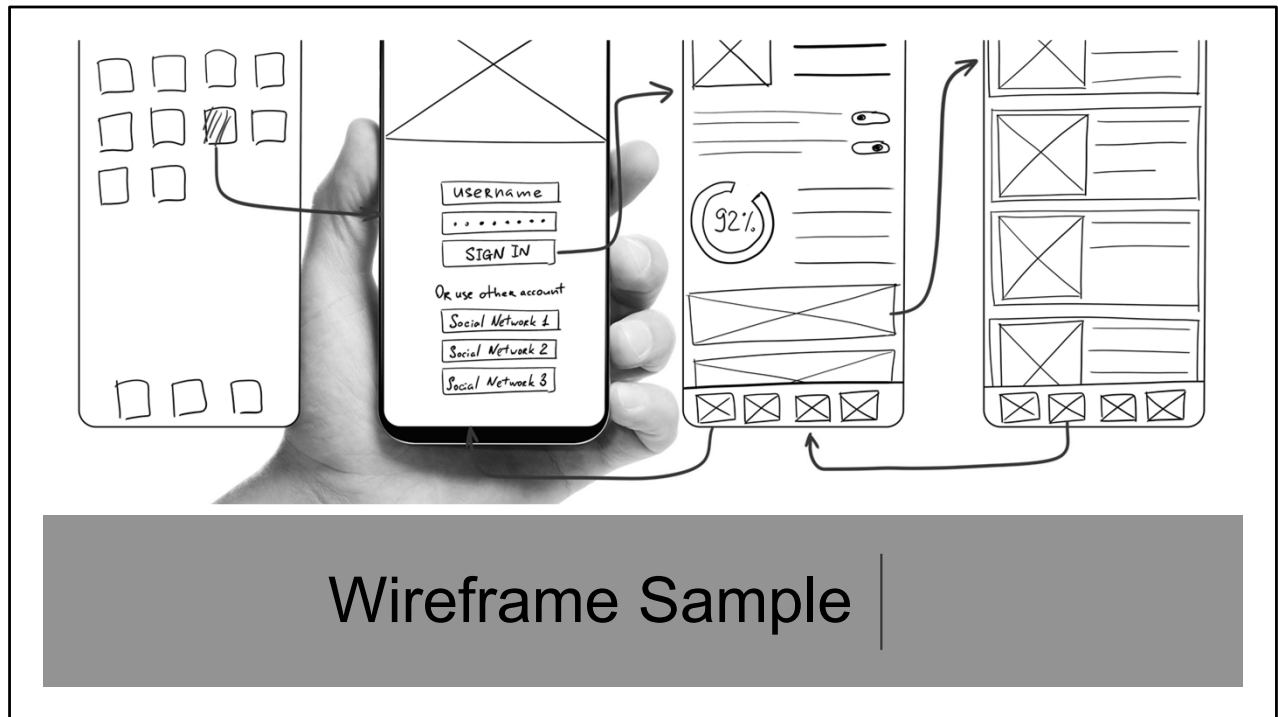


Introducing Wireframes

- Show don't tell
- Simple, quick diagrams showing solution
 - Element in UI
 - Element organization
 - Interface operation
 - User interaction with solution

Wireframes

- A quick mock-up of a product
- Could be screens and data flows between screens
- Ensures that everyone has the same understanding of the product
- A form of low fidelity prototyping
- Quick way to get feedback



User Personas

- Biographical sketches of key stakeholders
- Description of product users
- Somewhat grounded in reality
- Goal oriented
- Show tangible and actionable outcomes
- Focus on the users and who the users will be

Creating Personas

- Know the users and goals
- Precise is better than long (non-value add)
- Define a primary persona
- Focus on main usage
- Personas and user stories
 - As <persona> ,
 - I want <what?>
 - so that <why?>.

Contracts and Agile Approaches

Request for Proposal

A request for proposal is from the buyer to the seller

If the seller is to use a agile practices it must be defined in the request for proposal

The buyer may need to educate the vendor about agile practices

Agile project welcome change so this may be hard to do with contracts

Agile Constraints And Contracts

- Agile is flexible
- Contracts are not flexible
- Contracts are a form of a constraint
- Contracts are constrained by an offer and a consideration
- Collaboration over contracts

Agile Constraints And Contracts

- Agile projects constrain time and cost
- Agile project allow the scope to change
- Contracts typically constrain and balance time cost and scope

Considerations For Contracts

- Scope changes
- Priorities
- Time and cost

Introducing DSDM Contracting

- Dynamic Systems Development Method
- Grew out of RAD – Rapid Application Development in 90s
- Waterfall was too big, too slow
- RAD was not focused on quality
- DSDM wanted a balance

DSDM Strategy

- RAD benefits
 - Communication
 - Stakeholder engagement
 - Transparency
- Waterfall
 - Control
 - Quality

DSDM Contracting

- Follows DSDM approach
- Creates a fixed schedule, cost, and quality
- Follows agile framework

Graduated Fixed-Price Contract

- Both parties share some of the risk and reward
- If a vendor delivers on-time they get paid for their work at the hourly rate
- If the vendor delivers early they get paid for their work but at a higher hourly rate
- If the vendor delivers late they get paid for their work but at a lower hourly rate

Graduated Fixed-Price Benefits

- No sandbagging
- Celebrate success for working smart
- Initiative to work effectively
- Focus on results, not hourly rate

Fixed-Price Work Packages

The price of the work remains constant

Individual work packages are estimated for cost

Changes to the scope reflect a new estimate for those work packages

Customize Contracts

- The buyer and the seller can make any agreement they want
- Procurement is always tricky with agile projects

Gulf of Evaluation

- The difference between what is said and what is understood
- Intangible projects often experience this gulf
- What does DONE look like?

Value-based Analysis

Value-based Analysis

- Assessing and prioritizing the business value of work items
- Business benefit – cost
 - Business benefit equals \$8,000
 - Cost equals \$5,500
 - Value is \$2,500

More Value-based Analysis

- Will the item generate business value every week or month?
- A high business value item may be dependent on a low business value item

Value-based Decomposition

- Requirements elicitation
- Grouping of like features
- Breaking down of features
- Ranking of requirements
- Prioritize requirements into development

Creating A Product Box

- Imagine a product box or software
- Top three features
- Major functional elements
- Prioritization of features
- This is a visualization exercise



Coarse Grained Requirements

- Keeps the overall design balanced
- Delays decision on implementation details until the last responsible moment

Providing Estimate Ranges

Providing Estimate Ranges

- Not as precise as predictive planning
- More uncertainty and agile projects
- Include a range of variance
 - Between \$500,000 and \$550,000
 - Plus or minus 10%

Creating Agile Estimates

- Why is an estimate needed – to create a schedule and budget
- When does estimating happen – the last responsible moment and throughout the project
- Who does the estimating – team members estimate their own work
- How are estimates created – stages of sizing and planning; roll out an estimate cost may also be included
- How are estimates stated – always include a degree of uncertainty

Factoring Ideal Time

- Estimate as if there would be no interruptions
- Ideal time assumes all time in the estimate is for project work

Assumptions for Sizing and Estimating

- Details emerged as the project moves forward
- Plans are adjusted based on feedback
- Privatization happens throughout the project

Decomposing Project Requirements

- Breakdown of the project work
- Epics – large user stories that span one or more iterations
- Feature – attributes of the product
- User story – decomposition of a feature
- Task – smallest element of the decomposition

Voting on Requirements

Simple Voting

- Participatory decision approach
- The team votes for or against an idea by a show of hands

Thumbs Up, Down, or Sideways

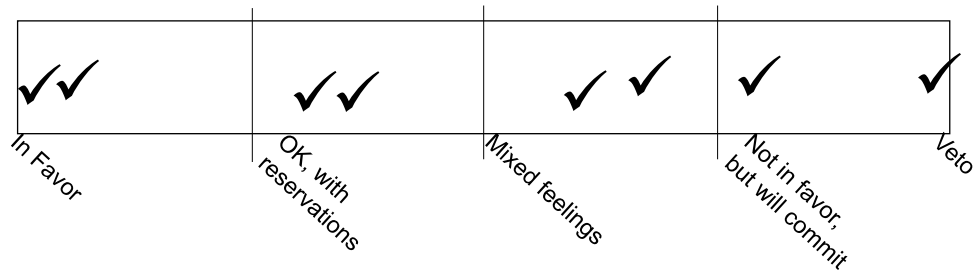
- Thumbs up and individual is for the decision
- Thumbs down and individual is against the decision
- Thumb sideways the individual is neutral or undecided



Fist of Five Voting

The number of fingers shown indicates degree of support

Highsmith Decision Spectrum



Planning the Product Release

Release Planning

Where is your product owner?

Do you have a prioritized backlog?

How will you size your items?

Who is coming?

Plan for logistics.

What about multiple/virtual teams?

Do I need help?

Release Planning Inputs

- Results of previous iterations and releases
- Feedback from stakeholders on the product, market situation, and deadlines
- Action plans and SMART goals from prior release and retrospective
- Items and defects to consider
- Development and architecture information
- Velocity from previous iterations or estimated
- Organizational and personal calendars
- Input from other teams and subject matter experts to manage dependencies

Release Planning Techniques

- Welcome, review purpose and agenda
- Product vision and roadmap
- Remind the team of the larger picture.
- Development status, state of architecture, results of previous iterations
- Discuss any new information
- Release name and theme
- Velocity in previous releases and iterations, or your estimated velocity
- Present the velocity to be used for release
- Release schedule and number of iterations

Release Planning Techniques

- Review key milestones and collaborative decision on timeboxes
- Issues and concerns
- Review and update the definition of Done
- Make any appropriate updates based on technology, skill, or changes in team membership since the last release.
- Stories and items from the backlog
- Present proposed backlog items to be considered for scheduling into this release
- Determine sizing values

Release Planning Techniques

- Agree upon sizing values
- Coarse sizing of stories
- Delivery team determines the size of items and splits items too large for iterations in the release
- Product owner and subject matter experts answer clarifying questions and elaborate acceptance criteria
- Map stories to iterations in the release
- New issues and concerns
- Dependencies and assumptions

Release Planning Techniques

- Commit!
- Fist of five voting
- Communication and logistics plan
- Parking lot
- Process the parking lot—all items resolved or turned into action items
- Action items and action plan
- Distribute action items to owners
- Retrospect the meeting
- Close

Story Maps

- High-level planning tool to map out project priorities
- Prioritized matrix of the features and user stories for the product being built
- Backbone – top level of the story map; essential functions for the system to work
- Walking skeleton – second row of the story map; smallest version of the system that will beat the customers both basic needs

Creating Product Roadmap

Creating Product Roadmap

- Visual depiction of product releases
- Primary items that will be included in each release
- Helps to check risk and viability

Product Roadmap

- Holistic view of the high-level functionality required to achieve the product vision
- Big picture view teams need to stay focused
- The initial cut at the product backlog

Wideband Delphi

- Rounds of anonymous estimates
- Helps to build consensus
 - Bandwagon effect – gathering around common viewpoint
 - Highest-paid person's opinion – HIPPO
 - Groupthink – making decisions to maintain group harmony

Defining Releases and Iterations

Defining Releases and Iterations

- Iterations are short, timeboxed periods of development
- Usually last two to four weeks
- Releases are the publishing of the software

Hosting A Release Planning Meeting

- All stakeholders represented
- Happens before each release
- The goal is to find which stories will be done in which iterations for the release
- Also defines future iterations for future releases

Hosting A Release Planning Meeting

- Assess the prioritize backlog
- Reviews story sizing
- Sort the stories by release
- Define the initial outline or road map for the release
- Slice the stories as needed for the plan release

Iteration Zero

- Sets the stage for development
- Typically no deliverables for the customer
- Prepares to do the work

Iteration H

- Hardening sprint
- Wrap up sprint
- Used to stabilize the code
- Document the product
- Compile final assembly
- Final testing

Architectural Spikes

- Proof of concept
- Timeboxed effort to test the approach

Risk-based Spike

- Short effort to investigate risk
- Reduce or eliminate through mitigation
- Good for new technology and early in the project

More about Slicing Stories

- Compound stories – includes other independent stories within
- Epics – one large complicated story; usually can't fit in one iteration

Estimating Story Sizes

Affinity Estimating

- Grouping items into similar categories or collections
- Group items based on story points
- Affinity estimating is like triangulation
- Allows the team to see the collection of user stories by points assigned

Utilizing Affinity Estimating

- Product owner
- Delivery team
- ScrumMaster

How to Do Affinity Estimating

- Stories on sticky notes
- Compare to proven reference stories
- Match stories comparable reference stories (affinity)

Relative Sizing for Story Points

- It's difficult to make absolute estimate
- Story points are points assigned to stories size
- Relative sizing assigns points to stories on a relative scale
- The team then decides how many points can be done in our generation

The Fibonacci Sequence

- Starting with zero the two numbers in the sequence are added together to get the next number
- 0 plus 1 equals one
- One plus one equals two
- 2 plus 1 equals 3
- 1, 2, 3, 5, 8, 13, or 21 points
- Only these numbers are assigned to user stories



Guidelines for Story Points

- The team owns the definition of their story points
- Story point estimate should be all inclusive
- Point sizes should be relative
- When disaggregating estimates the totals don't need to match
- Complexity work effort and risk are all included in the estimate

Sizing T-Shirts

- Small, medium, large, XL?
- Sizes are relative to one another
- Define the relationship between small and medium
 - Slightly larger?
 - Twice as large?

Planning for Iterations

Iteration Planning

- Meeting for and facilitated by the delivery team
- Confirms goal for the current iteration
- Discuss the user stories in the backlog
- Select the user stories for the iteration

Iteration Planning, continued

- Define the acceptance criteria and write the acceptance test for the stories
- Breakdown the use for stories in tasks
- Estimate the task

Velocity for Finding Estimates

- Base of the teams velocity more accurate estimates can be created
- Burn up charts
- Burndown chart

Verification and Validation

Examples Of Verification and Validation In XP

Pair programming

Unit testing

Customer collaboration

Stand up meetings

Acceptance testing

Iteration demonstrations

Product release

Frequent Verification and Validation

- Testing checkpoints and reviews
- Frequent verification and validation happen throughout project
- Build consensus between the project team and the project stakeholders

Unit Testing

- Unit – smallest testable portion of application code
 - Entire module
 - Function
 - Procedure
 - Basically, code fragments
- Automated process for testing
- Isolated testing for validity

Acceptance Testing

- Test to determine acceptability of results:
 - Specific requirement
 - Specification
 - Contract requirements

Exploratory Testing

- The tester aims to discover issues and unexpected behavior
- The tester explores the software
- This is in addition to scripted testing

Exploratory Testing Examples

- Tests created on the fly
- Investigation of system/application
- Improvement of test design – not automated
- Spontaneous test experiments
- Tester is in charge – not the script

Usability Testing

- How will a user respond to the system under realistic conditions?
- How easy is it to use the system?
- What improvements need to be made for usability?

User-Centered Interaction Design

- How users use the software
- Usability testing examples
 - Software
 - Food
 - Electronics
- Hallway testing – random people try the product
- Expert review – experts test the product

Continuous Integration System

Continuous Integration in Agile Projects

- Incorporate new and changed code into the code repository
- Small code commits
- Frequent integration
- Relies on automated tools to integrate code when new code is checked in

Continuous Integration System

Source code control system – version control

Build tools – build tools compile the code

Test tools – unit test to ensure functionality operates as expected

Scheduler or trigger – builds are launched on a schedule or based on conditions

Notifications – an email or instant message reporting on the results of a build

Why Continuous Integration?

- Early warning a broken conflicting or incompatible code
- Problems are fixed as they occur
- Immediate feedback
- Frequent unit testing define issues quickly
- Easy to reverse the code back to the last known good

Disadvantages Of Continuous Integration

- Set up time is lengthy
 - Often called Iteration 0
- Cost of a dedicated server
- Time required to build a suite of automatic tests

Exploring Test-Driven Development

Exploring Test-Driven Development

- Also called test first development
- Test are written before the code is written
 - Nunit
 - Junit
- Code is developed and edited until the code passes all tests
- Refactoring is the final step to clean up the code

Test-Driven Development

- Red – green – re-factor
- Red – green – clean
- Focus on the test first
- Early testing helps catch defects early in development
- Beware of developers writing their own test

Red-Green-Clean

- Create a unit tests that fails
- Create code that passes test
- Clean up the code

Acceptance Test Driven Development

- Testing focus is on business requirements
- Test represent the functionality the software is to have
- It's all about the desired behavior
- FIT – framework for integrated testing
 - Also called FitNessee
 - <http://www.fitnesse.org/>

Acceptance Test-Driven Development Cycle

- Discuss the requirements – developers as the product owner questions that are designed to gather acceptance criteria
- Distill test in a framework friendly format – gets the test ready to be entered into the acceptance test tool
- Developed the code and run the test – test initially fail because the code hasn't been written completely
- Demo – with automated acceptance testing scrips and demonstrations of the software

Stakeholder Engagement in Agile

Who are Project Stakeholders?

- People and groups impacted by the project
- Can impact the project
 - Customers
 - Project sponsor
 - Project leaders
 - Development team
 - Vendors
 - End users

Keeping Stakeholders Engaged

- Agile project manager works with stakeholders
- Not command and control
- Consider servant leadership
- Identify project stakeholders as early as possible

Educating Stakeholders for Agile Projects

- People new to agile projects need some basic education
- Address concerns directly with project stakeholders
- Explain the approach that will be used

Managing Stakeholder Engagement

- Short iterations keep stakeholders involved
- Reviews and demos show the results of the work
- Agile places value of work that is done
- Agile is naturally visible for project stakeholders

Incorporating Stakeholder Values

Incorporating Stakeholder Values

- Work is based on what the stakeholders value
- Engage the product owner to prioritize the backlog
- Work is executed by priorities
- The development team creates the highest priority items
- The development team delivers early value to the business
- Stakeholders are invited to planning meetings and retrospective

Principles of Stakeholder Engagement

- Get the right stakeholders
- Ensure a stakeholder participation
- Manage stakeholder interest
- Frequently discussed what done looks like
- Show progress to project stakeholders
- Openly discuss project estimates and projections

Defining Community Values

- Linked to Agile Manifesto
- Individuals and interaction
- Team and stakeholders are a sense of community
- What's important to these individuals and the group?
 - Respect for one another
 - Respect for time
 - Respect for end user
 - Respect for code and the process
 - Rules and team charter

Individuals and Interactions

- Respect for the worth of every person
- Truth in every communication
- Transparency of all data, actions, and decisions
- Trust that each person will support the team
- Commitment to the team and to the team's goals

Conflict and Stakeholder Engagement

- Conflict isn't always bad – positive conflict creates solutions
- List of impediments or problems and plan to tackle each
- Innovation occurs only with the free interchange of conflicting ideas
- Resolution of conflicting agendas by aligning around common goals
- People must agree on common goals and then work to become an effective team

Building Stakeholder Synergy

Building Stakeholder Synergy

- Stakeholder engagement
- Keeping stakeholders excited and involved
- Communication – truth in communication
- Agile involvement in process/project
- PMBOK Guide, 13.2

Tools and Techniques

- Expert judgment
- Data gathering
 - Benchmarking
- Data analysis
 - Assumptions and constraints
 - Root cause analysis
- Decision making
 - Prioritization/ranking
- Data representation
 - Mind mapping
 - Stakeholder engagement assessment matrix
- Meetings

Keeping Stakeholders Involved

- Framework of adaptive approach
- Daily standups
- Working with project team
- Regular communication
- Following rules of project

Daily Stand-Up Meeting Rules

- Stand up
- Three-question agenda
- Have PM tools handy
- Collaborative effort – speak to group
- Develop a routine

Creating A Project Tweet

- Project customers in the project team can work together to create a project tweet
- Describe the goal of the project in 140 characters or less
- This exercise defines a high-level description of the project
- Elevator statement

Leading Stakeholder Conversations

Leading Stakeholder Conversations

- Communication is paramount
- Push communication
- Pull communication
- Scheduled interactive communication
- Transparency

Good News, Bad News, and All News

- Transparency is paramount for exam
- Don't hide bad news – go to the problem
- Confront issues
- Stakeholders are partners in the project

Stakeholder are Involved

- Design documents
- Software and prototype demonstrations
- Retrospectives and capturing lessons learned
- Providing information for unclear requirements
- Test scenarios and test cases for UAT
- Performing UAT
- Defining transition requirements
- Preparing the organization for change

Create the Project Vision

Create the Project Vision

- Develop the project vision based on
 - Key goals
 - Customer needs, wants, and expectations
 - Competition or similar products (differentiate solution)
- Create the Vision Statement
 - Write in present tense (act as if it exists already)

Create the Project Vision

- Validate and revise
 - Compelling for stakeholders
 - Organization's goals and strategy
- Consider roles in project
 - Stakeholders: accomplishes goal
 - Team: understand the end results
 - Project manager: remove impediments
- Final Version
 - Visible to all in project
 - Reference throughout project

Failing Fast

- Failing fast means failing early and cheaply
- Good way to discover misunderstandings
- Ensures the project team understands what stakeholders want

Creating an Agile Charter

Creating an Agile Charter

- Agile charters authorize the project and the project manager
- Agile charters are from the project sponsor
- Could be lightweight or very detailed
- Acknowledge change is likely in the actual project

Agile Project Charters are Different

- Traditional charters are very specific
- Agile charters are broad and high level
- Agile charters define:
 - Who will be engaged
 - What is the project about
 - Where will the project take place
 - When will the project start and end
 - Why this project being chartered
 - How the goals of the project be achieved

Agile Charter Elements

- Vision Statement
- Team rules (constitution)
- Code of Conduct
- Communication rules
- Definition of Done/Success factors

An Agile Project Charter

- For an agile project, the team needs the project vision or purpose and a clear set of working agreements.
- An agile project charter answers these questions:
 - Why are we doing this project?
 - Who benefits and how?
 - What does done mean for the project?
 - How are we going to work together?

Agile Charter

- Value is creation (not a template)
- Single page (no value in documentation)
- Whole team approves the charter (consensus)
- Define success criteria (outcome)

An Agile Team Charter

- Some teams benefit from a team chartering process.
- Here are some chartering ideas for team members to use as a basis for their social contract:
 - Team values
 - Working agreements – what does ready and done mean?
 - Ground rules – one person talking
 - Group norms – meeting times/duration
 - The servant leader together with the team
 - may decide to address other behaviors.

Assignment: Creating a Charter

Agile Communication Consideration

Plan Communications Management

- Communication requirements analysis
- Communication technology
- Communication model
- Communication methods
- Meeting
- Create the communications management plan
- Update project documents

Manage Communications

- Communication technology
- Communication models
- Communication methods
- Information management systems
- Performance reporting
- Communicate with stakeholders
- Update the project management plan project document
organizational process assets

Control Communications

- Information management systems
- Expert judgment
- Meetings
- Work performance information
- Change request
- Update the project management plan project documents and organizational process a set

Face-to-face Communication

- Face-to-face communication is preferred
- Highest bandwidth of all communication types
- Most effective communication

Two-way Communication

- Dispatching model – top down communication
- Collaborative model – interactive communication between sender and receiver

Agile Knowledge Sharing

Knowledge Sharing

- Knowledge sharing is critical for agile projects
- Share information with everyone
- Collective code ownership means any developer can edit any code at any time
- Agile practices promote knowledge sharing:
 - Kanban boards
 - Information radiators
 - Personas
 - Wireframes

Communicating And Agile Projects

- Low-tech high-touch tools
- Stand up meetings
- Osmotic communications
- Tacit knowledge

What is an Information Radiator?

- Highly visible displays of information
- Large graphs or charts that summarize project data
- Out in the open and easily accessible
- Also known as visual controls

What's on an Information Radiator?

- Features delivered versus features remaining
- Who is working on what
- Current iteration features to be created
- Velocity and defect measurements
- Retrospective outcomes
- Threats and issues for the project
- Burn up and burndown chart
- Story maps

Social Media and Agile Projects

Social Media and Agile Projects

- Remote workers staying in touch
- Facebook
- Twitter
- Web collaboration tools
- Non co-located teams
- Consider the sensitivity of the project information

Collaboration Tools Consideration

- IP telephone
- Skype
- GotoMeeting
- Adobe Connect
- Google Hangout
- Messenger

Agile Collaboration

Collaboration is Key in Agile Projects

- Customer collaboration over contract negotiation
- Business people and developers must work together daily throughout the project

Benefits of Collaboration

- Generates wiser decisions
- Promotes problem solving
- Promotes action
- Build social capital
- Ownership of collective problems

Engaging People in Agile Projects

- Engagement creates better ideas and put some conversations
- Active problem solving instead of command and control
- Taking action rather than being passive
- Collective ownership of ideas
- Motivates and engages the project team
- Shifts the power downward

Green and Red Zones

Green Zone versus Red Zone

- Takes responsibility
- Responds non-defensively
- Not easily threatened
- Build mutual success
- Seeks solutions
- Uses persuasion
- Firm, but not rigid
- Thinks both short-term and long-term
- Considers other points of views
- Welcomes feedback
- Considers conflict to be natural
- Speak calmly and directly about difficult issues
- Accept responsibility
- Seeks excellence

Green Zone Versus Red Zone

- Blames others
- Responds defensively
- Feels threatened or wrong
- Triggers defensiveness
- Holds grudges
- Shame, blame, and accusations
- Binary thinking
- Short-term advantage
- Feel victimized
- Doesn't seek feedback
- Must win at any cost
- Is rigid and reactive
- Creates a climate of antagonism
- Disapproval and contempt
- Sees others as the enemy
- Does not listen effectively

Interpersonal Skills for Agile Projects

Interpersonal Skills for Agile Projects

- Emotional intelligence
- Active listening
- Facilitation techniques
- Negotiation
- Conflict resolution
- Participatory decision making

What is Emotional Intelligence?

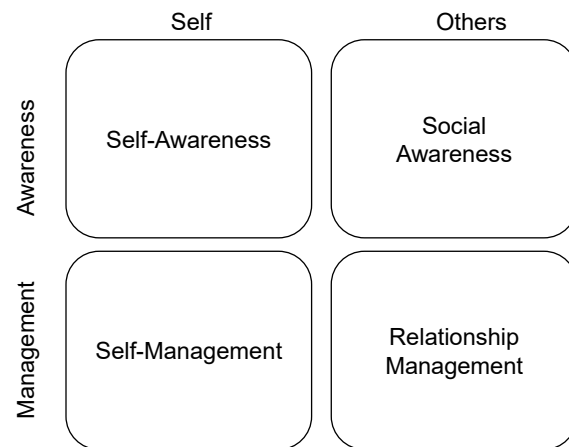
- Emotional intelligence affects every aspect of your life
- Understanding the root causes of emotions
 - Identify who you are
 - Better interact with others

With Emotional Intelligence being a fairly new branch of psychology, its definition can be found in various theories and models. We are presenting a definition influenced and popularized by Daniel Goleman's 1995 book *Emotional Intelligence*.

Quadrants Of Emotional Intelligence

- Self-management
 - Self control
 - Conscientiousness
 - Adaptability
 - Drive and motivation
- Self-awareness
 - Self confidence
 - Emotional self-awareness
 - Accurate self-assessment
- Social skills
 - Influence
 - Inspirational leadership
 - Developing others
 - Teamwork and collaboration
- Social awareness
 - Empathy
 - Organizational awareness
 - Understanding the environment

Quadrants of Emotional Intelligence



Negotiation and Collaboration

Negotiation

- Negotiations happen throughout the project
- Consider the priorities of user stories
- Avoid a zero-sum games where only one person wins
- Healthy negotiations allow for give and take

Customer Collaboration

- Accept (do nothing)
- Avoid (create a work-around)
- Ameliorate (reduce the impact)
- Cover (make it invisible to the user)
- Resolve (eliminate it once for all)

Story Negotiation

- Separate people and problems
- Focus on shared interests
- Create options
- Use objective criteria

Resolving Conflicts in Agile Projects

Conflict Resolution

- Differences of opinions and competing interest
- Some conflict is healthy

Level	Name	Characteristic	Language
One	Problem to solve	Information sharing and collaboration	Open in fact based
Two	Disagreement	Personal protection trumps resolving the conflict	Guarded and closed
Three	Contest	Winning trumps resolving the conflict	Includes personal attacks
Four	Crusade	Protecting one's own group becomes the focus	Ideological
Five	World War	Destroy the other	Little or non-existent

Conflict Resolution Methods

- Withdraw/Avoid
- Smooth/Accommodate
- Compromise/Reconcile
- Force/Direct
- Collaborate/Problem Solve

Convergent And Shared Collaboration

- Convergent – participating decision making models in for conversions for collective agreements
- Shared collaboration they share the decision-making process fairly

Agile Teams Considerations

Generalizing Specialist Attributes

- One or more technical specialties
- At least general knowledge of software development
- At least a general knowledge of the business domain
- Seeks to gain skills in existing specialties and other areas

Characteristics Of High-performing Teams

- Create a shared vision for the project team
- Set realistic goals
- Limit team size to 12 or fewer people
- Build a sense of team identity
- Provide strong leadership

Eight Characteristics of High-Performance Teams

- Self-organizing
- Empowered
- Believe that as a team they can solve any problem
- Committed to team success
- Owns its decisions and commitments
- Motivated by trust
- Consensus driven
- Participate in constructive disagreement

Self-organizing Teams

- Not command and control
- Can use their own knowledge to organize work
- Structure that work based on iteration goals
- Responsibility delegated to the team

Self-directing Teams

- Empowered to work collectively
- Make local decisions
- Estimate and decide the project work
- Make mistakes and learn from mistakes

Dreyfus Model Of Adult Skill Acquisition

- Novice follow the rules they've been given and make analytical decisions
- Advanced beginner still following the rules but based on experience better understand the context of the rules
- Competent determining which rules are best for each situation
- Proficient actively choosing the best strategy rather than simply relying on the rules
- Expert decision-making becomes intuitive

Shu-ha-ri Model Of Skill Mastery

- Shu – start by following the rules
- Ha – once the team has mastered the guidelines they can move away from them and work intuitively
- Ri – the team reaches full mastery and can transcend the rules

Five Dysfunctions Of A Team

- Absence of trust
- Fear of conflict
- Lack of commitment
- Avoidance of accountability
- Inattention to results

Creating a Team Space

Creating a Team Space

- Location for team members in the same space
- Also known as the war room
- Visible information radiators for project metrics
- Lots of white boards and task boards

Caves and Commons

- Caves are private spaces for phone calls or one-on-one conversations
- Commons are the primary work area

Managing Virtual Teams

- Consider different time zones
- Consider different cultures
- Different communication styles
- Different native languages

Managing Distributed Teams

- Distributed teams are virtual teams
- Short iterations help collaboration in coordination
- Distributed teams is not the same as outsourcing
- Distributed teams face more of a challenge in storming and norming
- The project manager may need to introduce controversial or difficulties of the work early in the project
- A face-to-face kickoff is often needed

Digital Tools For Distributed Teams

- Video conferencing
- Interactive whiteboard
- Instant messaging
- Presence-based application
- Electronic task for storyboards
- Web-based meeting facilitators
- Virtual card wall
- Wiki site

Completing Team Assessments

Completing Team Assessments

- Use velocity to measure individual performance
- Use a task matrix for individuals
- Create goals for each individual
- Compare goals achieved per iteration

Team Self Assessments

- Team self assessment scoring model
 - Thinking
 - Collaborating
 - Releasing
 - Planning
 - Developing

Problem Detection and Resolution

Four Themes In Problem Detection and Resolution

- Understanding problems
- Detecting problems
- Managing threats and issues
- Solving problems

Risk Management

- Risk are uncertain events or conditions
- An agile projects risk are always negative
- Probability and impact
- Risk identification and tracking

Issues and Risk

- Risk are uncertain events they've not yet happened
- Issues are risk events that have occurred

Truly Understand Problems For Resolution

- Problems can mushroom
- Problems can have ripple effects
- What happens in one area of the project affect all other areas of the project

Financial Impacts Of Problems

- The longer a defect is left unaddressed, the more expensive it will be to fix
- It is like going to the dentist
- The longer you wait the more expensive it will be

Detecting Problems and Defects

- Escaped defects
- Fix defects upon discovery in each iteration
- Regression testing on all functionality – periodically
- Demonstrating user story solution
- UAT and end-to-end testing
- Goal: discover and fix defects before release

Start With the Daily Standup

- Are there any problems or impediments to the team members?
- This is the first step to detecting problems

Reviewing Technical Debt

- Backlog of work caused by lack of regular clean-up maintenance and standardization
- Refactoring solves technical debt
- Red green refactor
- Refactoring cleans up and standardizes code to make it easier to support
- Refactoring should be included in estimates

Reviewing Technical Debt

- Not unlike borrowed money
 - Accumulates interest to be repaid later on
 - Takes time to correct the debt
- Can be caused by many factors:
 - Poor upfront definition
 - Pressure for value delivery
 - Lack of documentation
 - Parallel development

Reviewing Lead Time and Cycle Time

- Lead time is how long something takes to go through the entire process
- For example, from concept design to shipping
- Cycle time is a subset of lead time
- Cycle time is how long something takes to go through a part of the process
- For example, from coding to testing

Project Cycle Time

- The project duration is the cycle time for the entire project
- Average amount of work the team can get done in a time.
- Productivity is the rate of efficiency at which the work is done
- Productivity could be the amount of work done per team member

Looking at Defect Cycle Time

- Defect cycle time is the amount of time between when the defect was discovered and when the defect was fixed
- The longer the defect cycle time typically the more expensive the defect

Defect Rates

- Defects that slip by the testing team are called escaped defects
- Escape defects are the most expensive to fix
- The defect rate measures the frequency of defects found
- An increase in escape defects signals a problem with a process

Assignment: Case Study - Cycle Time and Lead Time

Variance Analysis

Variance Analysis

- The difference between what was planned and what was experienced
- Cost variance
- Schedule variance
- Other tracking items

Causes Of Variance

- Average day to day differences
 - Good days and bad days
 - Ups and downs
- Special causes of variance
 - Something unusual that causes a problem
 - Power went out for two days
 - Three team members caught the flu

Defining Trend Analysis

- Time series data
- Comparing same data in each time period
- Identify relationships
- Predict future performance

Relying on Trend Analysis

- Measurements of past experiences are called lagging metrics
- Leading metrics provide a view into the future
- Trend analysis aims to predict performance or problems

Working With the Risk-adjusted Backlog

Working With the Risk-adjusted Backlog

- Risk is considered anti value
- Goal is to attack high-risk items early in the project
- Items with the greatest value in greatest risk move to the top of the backlog

Backlog Grooming for Risk

- Stories are ranked based on business value and risk level
- The ranking is subjective often based on gut feeling or preference
- The return on investment for the project can be broken down per item
- The business representatives assign the ROI to each item in the backlog

Finding the Expected Monetary Value

- Expected monetary value is the worth of a risk event
- $EMV = \text{risk impact} \times \text{risk probability}$
- $EMV = \$45,000 \times 30\%$
- $EMV = \$13,500$
- This is done for each risk in a probability impact matrix

Probability-Impact Matrix

Risk event	Probability	Impact	Ex\$V
A	.60	-10,000	-6,000
B	.20	-75,000	-15,000
C	.10	25,000	2,500
D	.40	-85,000	-34,000

Contingency reserve = \$52,500

Determining Risk Severity

Risk Severity

- Instead of risk probability and dollar amounts
- Probability and impact uses a simple scale
- For example low medium high
- Risk severity = risk probability x risk impact
- Risk severity = 3 x 1

Probability-Impact Matrix

- Cardinal scale
- Risk exposure
- Sum of contingency reserve
- “Hedging bets”

Probability-Impact Matrix

Risk event	Probability	Impact	Ex\$V
A	.60	-10,000	-6,000
B	.20	-75,000	-15,000
C	.10	25,000	2,500
D	.40	-85,000	-34,000

Contingency reserve = \$52,500

Problem Solving is Continuous Improvement

Problem Solving is Continuous Improvement

- Problem solving games to fix the problem before it happens
- Consider daily standup
- Iteration reviews and retrospectives
- Sprint planning sessions

Considerations for Team Engagement

- Involve the team where it can be most helpful
- Solve real problems only
- Team cohesion is necessary
- Check in after project changes
- Be sure to follow-through

Some Problems can't be Solved

- Even with team engagement some problems won't find a solution
- Work around these problems
- Track and monitor the problems

Continuous Improvement Tasks

Continuous Improvement Tasks

- Periodically review and tailor the process
- Improve team processes through retrospectives
- Seek product feedback via frequent demonstrations
- Create an environment for continued learning
- Used values dream analysis to improve processes
- Spread improvements to other groups in the organization

Continuous Improvement and Quality

- Quality assurance is prevention driven
- Quality assurance is planning for quality
- Quality control is inspection driven
- You cannot inspect quality into a product

Leading Continuous Improvement as a Process

- Ongoing effort to improve processes
- Effectiveness and/or efficiency to meet objectives
- WIP and Velocity
- Realistic goals

Lessons Learned

- Lessons learned are captured in each iteration
- Allows lessons to be applied in next iterations
- Lessons are repeated until they are learned

Kaizen

- Japanese word meaning change for the better
- Small incremental steps for improvement
- Plan Do Check Act
- Plan, develop, evaluate learn

Methodology Success Patterns

- Interactive face-to-face communication is the cheapest and fastest channel for exchanging information
- Excess methodology weight is costly
- Larger teams need heavier methodologies
- Projects with greater criticality require greater ceremony

Methodology Success Patterns

- Feedback and communication reduce the need for intermediate deliverables
- Discipline, skills, and understanding counter process formality and documentation
- Efficiency is expendable on bottleneck activities

Project Pre-mortem

Project Pre-mortem

- Aims to find failure points before they happen
- Imagine the failure
- Generate the reason for failure
- Consolidate the list
- Revisit the plan

Hosting the Pre-mortem

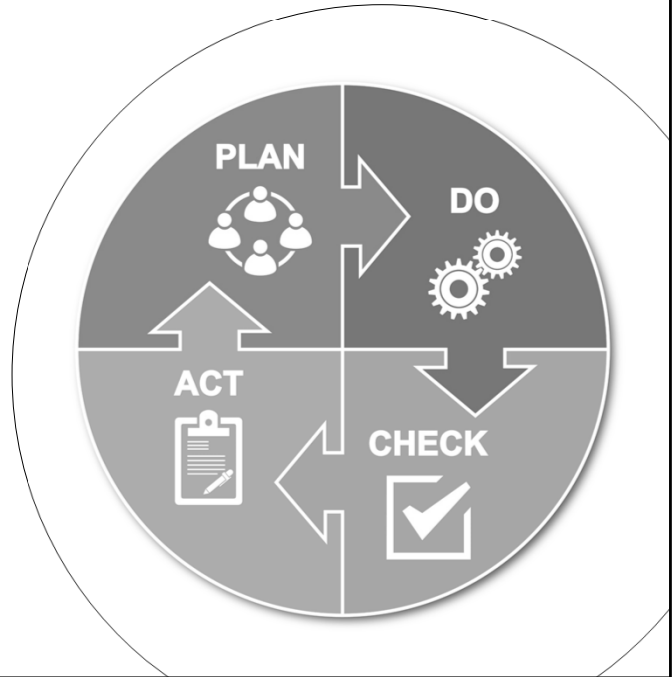
- Discuss what might go wrong
- Typically after plan created
- People create a prioritized list of possible wrong outcome
- In turn, each person gives one reason until all reasons addressed

Product Reviews

Product Reviews

- Product feedback
- Retrospective and demos

PDCA Cycle



Product Feedback Loops and Learning Cycles

- Does it meet the customers needs and expectations?
- Does it work and all conditions?
- Did we break anything while building this?
- How can we improve efficiency?
- How can quality be improved?
- How can we share lessons learned?

Product Feedback Methods

- Prototypes
- Simulations
- Demos

Product Feedback Strategies

- Don't talk to all users
- Have ongoing feedback mechanisms
- Identify vocal minority
- Don't make assumptions about feedback content

Approved Iterations

- Iterations and sprint reviews
- Held at the end of a sprint
- Demonstrates the new incremental build
- Business partner approved work
- Next sprint may begin

Incremental Delivery for Value

- Reduces risk
- Delivers value fast and first
- Retrospectives for process improvements

Approved Iteration Considerations

- Defined iteration then review
- Product increment compared to backlog
- Approved iterations can be linked to payments

Retrospective for People Improvement

Retrospective for People Improvement

- What is going well?
- What areas could use improvement?
- What should we be doing differently?

Why You Need Retrospectives

- Improve productivity
- Improved capabilities
- Quality improvement
- Capacity improvement

Setting the Stage for a Retrospective

- Encourage participation
- Set the ground rules
- Define what people want from the retrospective
- Have people checking in with one or two words
- Ask the team to commit to focus
- Explorer shopper/vacationer/prisoner
- Working agreements for the retrospective

Gather Data in the Retrospective

- Use techniques to extract data
- Get people involved in contributing
- Facilitate the meaning for contributions

Methods For Gathering Data in the Retrospective

- The team creates a timeline
- Triple nickels – groups spending five minutes on 5 ideas 5 times
- Color-coded dots – used color-coded dots to track your energy was high and low throughout the duration
- Mad, sad, or glad – track emotions throughout the timeline

Methods For Gathering Data During the Duration

- Satisfaction histogram – a bar chart showing satisfaction about particular areas or issues
- Team radar – an assessment of performance improvement
- Locates strengths - what went well, or not well, in the iteration
- Like to like – compares reactions to different iteration events

Generating Insights

- Evaluate the data
- Create insights based on the data
- Done with the project team

Brainstorming

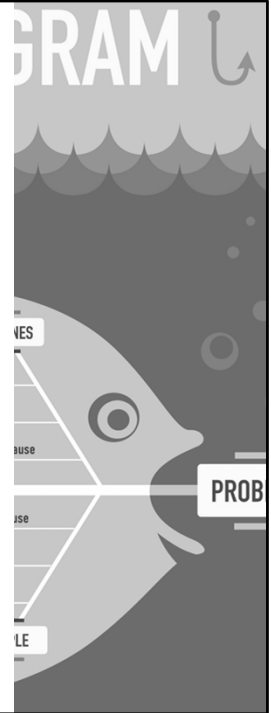
- Quiet writing
- Round robin
- Free for all

Five Whys

- Cause and effect exercise
- Small groups
- Ask why five times
- The goal is to find root cause

Fishbone Analysis

- Also known as a cause-and-effect diagram
- Also known as an Ishikawa diagram



Decide What To Do

- Create an action plan
- Short subjects – keep, drop, add
- Smart goals – specific, measurable, attainable, relevant, timely
- Circle of questions – each person ask a question on how to improve one of the issues; the next person of the circle answers the question
- Retrospective planning game – how to reach process improvement goals

Close the Retrospective

- Plus / delta – what to do more of and what to do less of
- Help, hindered, hypotheses – feedback on the retrospective
- Return on time invested – the team discuss the benefits of the retrospective and gives a great
- Appreciation – the team gives appreciation to other team members based on efforts from the last iteration